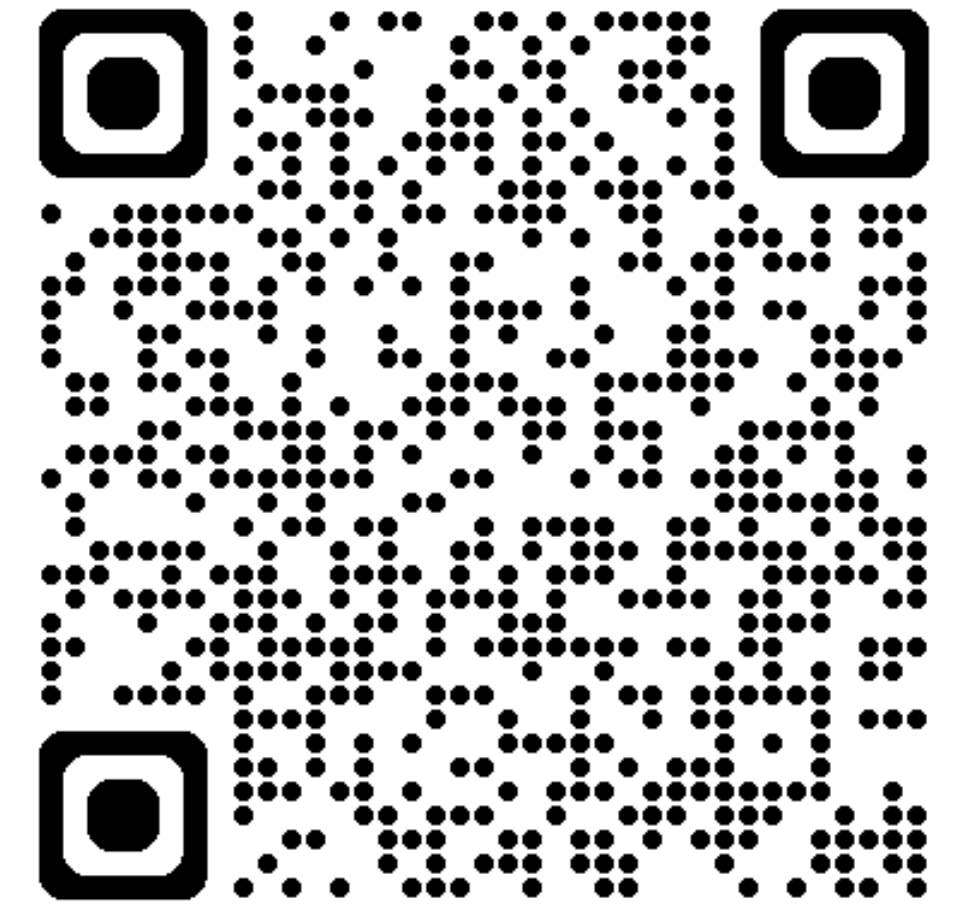


Übungsstunde 9

Heute:

- Repetitions Quiz
- Hashing
- Quadtrees
- Stack& Queue



Repetitions Quiz

basierend auf die Zwischenfrage in der letzten Woche

```
def transform_data(number):  
    #print(id(number), 'in function call, before rising by 10')  
    number +=10  
    #print(id(number), 'after +=10')  
    number= 1000  
    number -=100  
  
    return number  
  
dreissig = 30  
#print(id(dreissig), 'after init')  
result = transform_data(dreissig)  
|  
# Was ist der Wert von info?  
print(dreissig,result)
```

nun mit einem int

```
def transform_data(data):  
    data['status'] = 'processed'  
    data = {'status': 'new', 'value': 42}  
    data['value'] += 1  
    return data  
  
info = {'status': 'pending', 'value': 10}  
result = transform_data(info)  
  
# Was ist der Wert von info?  
print(info)
```

✓ 0.0s

`{'status': 'processed', 'value': 10}`

reminder: letzte Woche

tipp: immutable/mutable?

Repeitions Quiz

basierend auf die Zwischenfrage in der letzten Woche

```
def transform_data(number):  
    #print(id(number), 'in function call, before rising by 10')  
    number +=10  
    #print(id(number), 'after +=10')  
    number= 1000  
    number -=100  
  
    return number  
  
dreissig = 30  
#print(id(dreissig), 'after init')  
result = transform_data(dreissig)  
|  
# Was ist der Wert von info?  
print(dreissig,result)
```

nun mit einem int
lsg: 30,900

tipp: immutable/mutable?

```
def transform_data(data):  
    data['status'] = 'processed'  
    data = {'status': 'new', 'value': 42}  
    data['value'] += 1  
    return data  
  
info = {'status': 'pending', 'value': 10}  
result = transform_data(info)  
  
# Was ist der Wert von info?  
print(info)  
  
✓ 0.0s  
{'status': 'processed', 'value': 10}
```

reminder: letzte Woche

Refresher Laufzeiten

korrekte Laufzeit finden, ohne ZF!

$$\sqrt{\sum_i^n i^3}$$

$$\sum_i^n \log(i)$$

* star bei $i = 1$, bei beiden sigmas

Refresher Laufzeiten

korrekte Laufzeit finden, ohne ZF!

$$\sqrt{\sum_i^n i^3}$$

laufzeit: n^2

$$\sum_i^n \log(i)$$

laufzeit: $n \cdot \log(n)$

* star bei $i = 1$, bei beiden sigmas

Welcher Sortieralgorithmus ist das?

Zur Auswahl: BubbleSort, MergeSort, InsertionSort,

[9, 5, 1, 6, 8]

1 [5, 9, 1, 6, 8]

2 [1, 5, 9, 6, 8]

3 [1, 5, 6, 9, 8]

4 [1, 5, 6, 8, 9]

*zu sehen: iteration und den zustand der Liste

Welcher Sortieralgorithmus ist das?

Zur Auswahl: BubbleSort, MergeSort, InsertionSort,

[9, 5, 1, 6, 8]

1 [5, 9, 1, 6, 8]

2 [1, 5, 9, 6, 8]

3 [1, 5, 6, 9, 8]

4 [1, 5, 6, 8, 9]

Lösung: Insertion sort, es ist zu sehen wie der Sortierte bereich wächst.

Was ist die Ausgabe?

```
liste_a = [1, 2, 3, 4]  
liste_b = ['A', 'B', 'C']  
ergebnis = list(zip(liste_a, liste_b))  
print(ergebnis)
```

✓ 0.0s

Was ist die Ausgabe?

```
liste_a = [1, 2, 3, 4]
liste_b = ['A', 'B', 'C']
ergebnis = list(zip(liste_a, liste_b))
print(ergebnis)
```

✓ 0.0s

(ergebnis =) [(1, 'A'), (2, 'B'), (3, 'C')]

```
def op_a(arg):
    return 2*arg

op_b= lambda u: sum(len(v) for v in u)

print(op_b(op_a(ergebnis)))
```

✓ 0.0s

Was ist die Ausgabe?

```
liste_a = [1, 2, 3, 4]
liste_b = ['A', 'B', 'C']
ergebnis = list(zip(liste_a, liste_b))
print(ergebnis)
```

✓ 0.0s

(ergebnis =) [(1, 'A'), (2, 'B'), (3, 'C')]

```
def op_a(arg):
    return 2*arg

op_b= lambda u: sum(len(v) for v in u)

print(op_b(op_a(ergebnis)))
```

✓ 0.0s

12, da $2 * (3 * 2)$

Was ist die Ausgabe?

```
zahl = 31

def func(arg):
    return arg
    if arg%2==0:
        return arg/2
    else:
        return func(arg-1)

print(func(zahl))
```


Datenstrukturen aus Informatik I

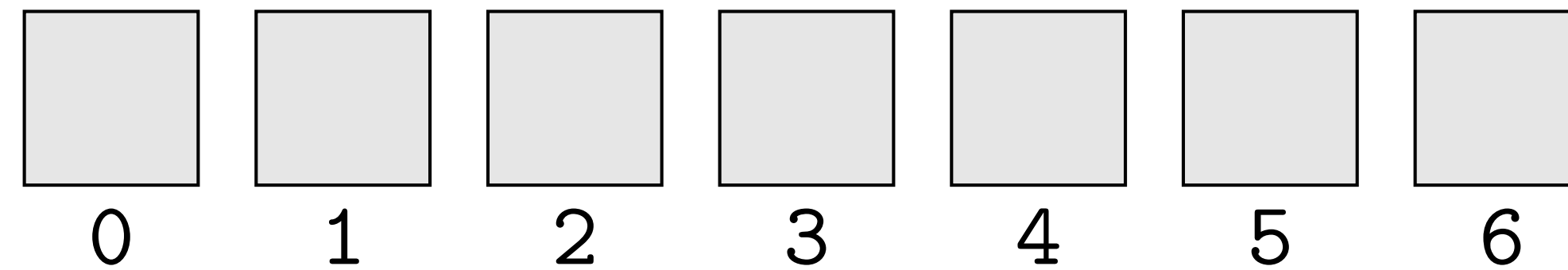
- Vektor **vector**
- Verkettete Liste **llvec**
- Set (Menge) **unordered_set<T>**
- Stack (Stapel, LIFO) **stack**
- Baum **tnode**

Informatik II bis jetzt:

- Liste **list**
- Set (Menge) **set**
- Dictionary **dict**

⇒ Potpourri von Datenstrukturen: Wähle die beste abhängig vom Kontext!
Heute werden wir die dafür nötigen Unterscheidungen lernen.

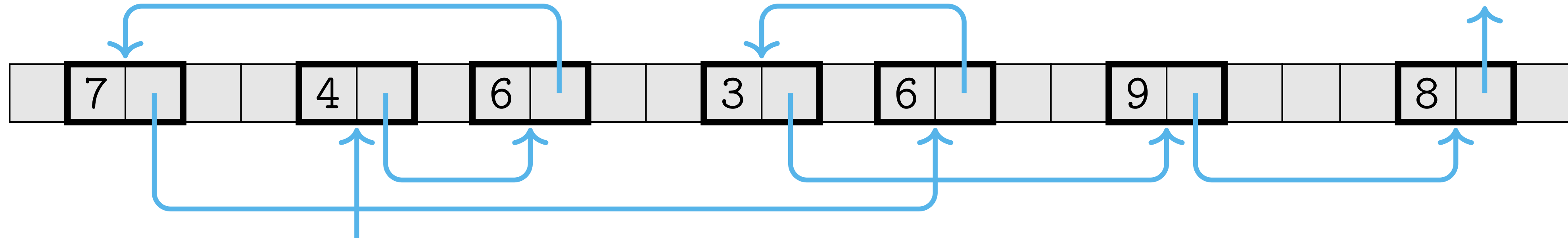
Liste



Operation	Laufzeit
Index-Zugriff	$\mathcal{O}(1)$
Suche (in)	$\mathcal{O}(n)$
Sortierte Suche	$\mathcal{O}(\log n)$
Einfügen	$\mathcal{O}(n)$
Entfernen	$\mathcal{O}(n)$

Sortierte Suche wird in Übung 6 behandelt

Verkettete Liste (Linked List)



Vorteil: einfügen,
entfernen alle $O(1)$,
da im Speicher
nichts
herumgeschoben
werden muss

Operation	Laufzeit
Sortierte/Unsortierte Suche	$O(n)$
Einfügen nach Knoten	$O(1)$
Entfernen nach Knoten	$O(1)$
Einfügen nach Wert	$O(n)$
Entfernen nach Wert	$O(n)$

Balancierter Binärer Suchbaum

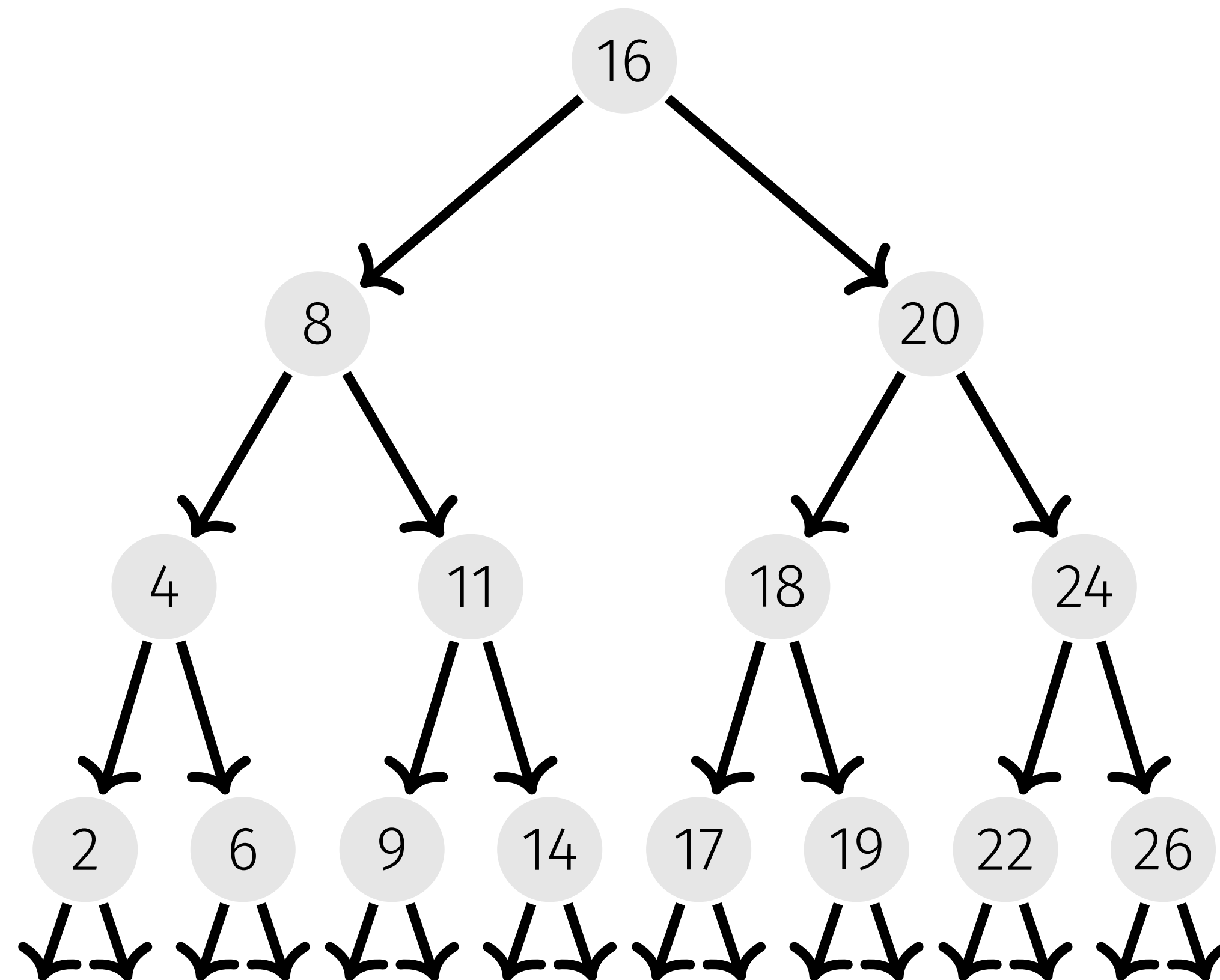
Für jeden Knoten

■ **linker Teilbaum:**
kleinere Schlüssel

■ **rechter Teilbaum:**
grössere Schlüssel

balanciert: Höhe $\mathcal{O}(\log n)$

Operation	Laufzeit
Suche	$\mathcal{O}(\log n)$
Einfügen	$\mathcal{O}(\log n)$
Entfernen	$\mathcal{O}(\log n)$



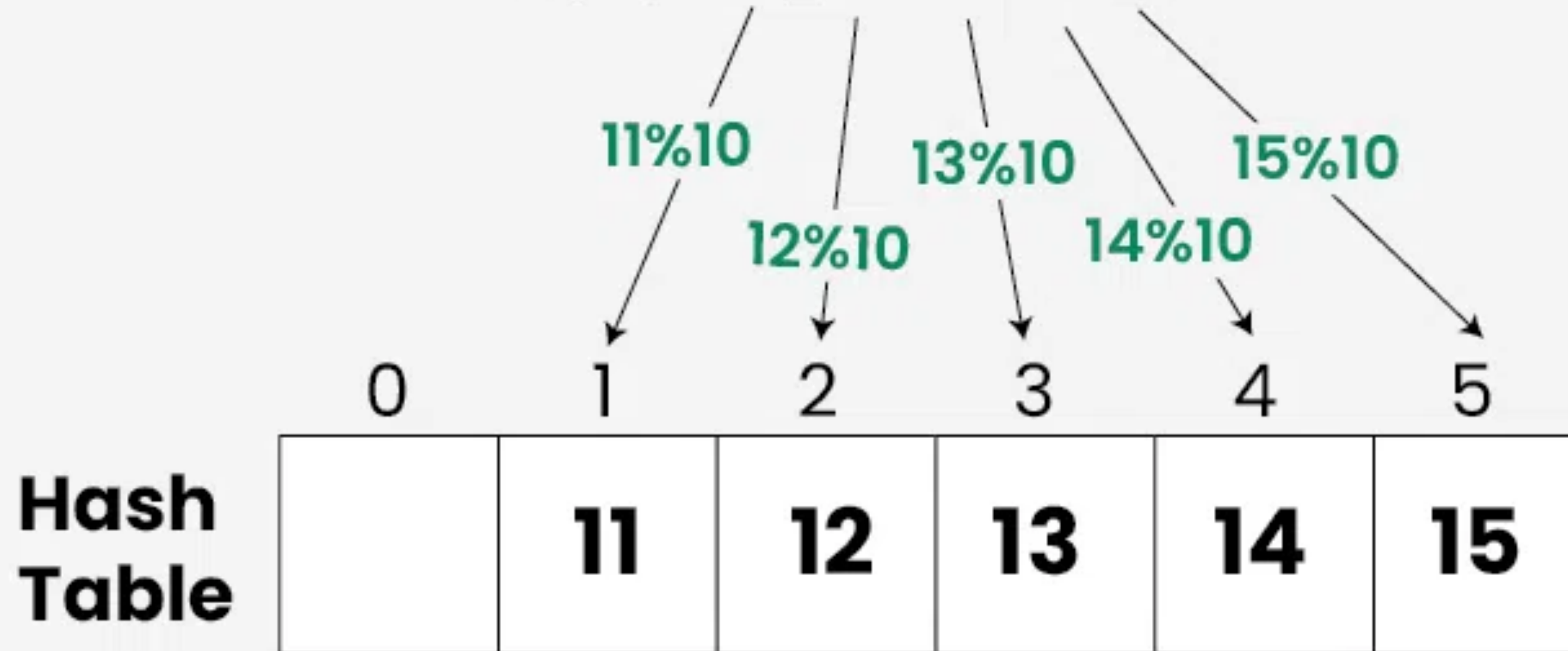
repetition: weshalb ist die Suche $n \cdot \log(n)$, wie wird im BST gesucht?

Hash-Tables

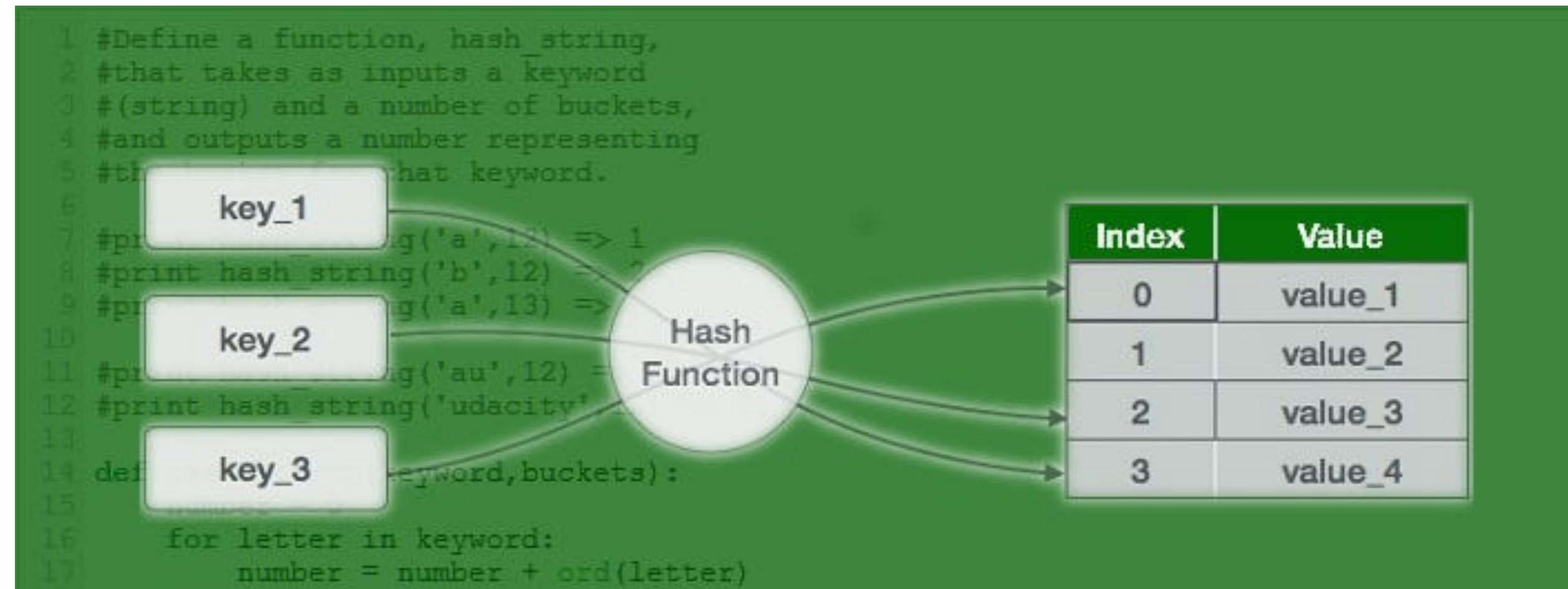
Hashing allgemeines Prinzip:

List = [11, 12, 13, 14, 15]

$$H(x) = [x \% 10]$$



Charakteristiken Hashing



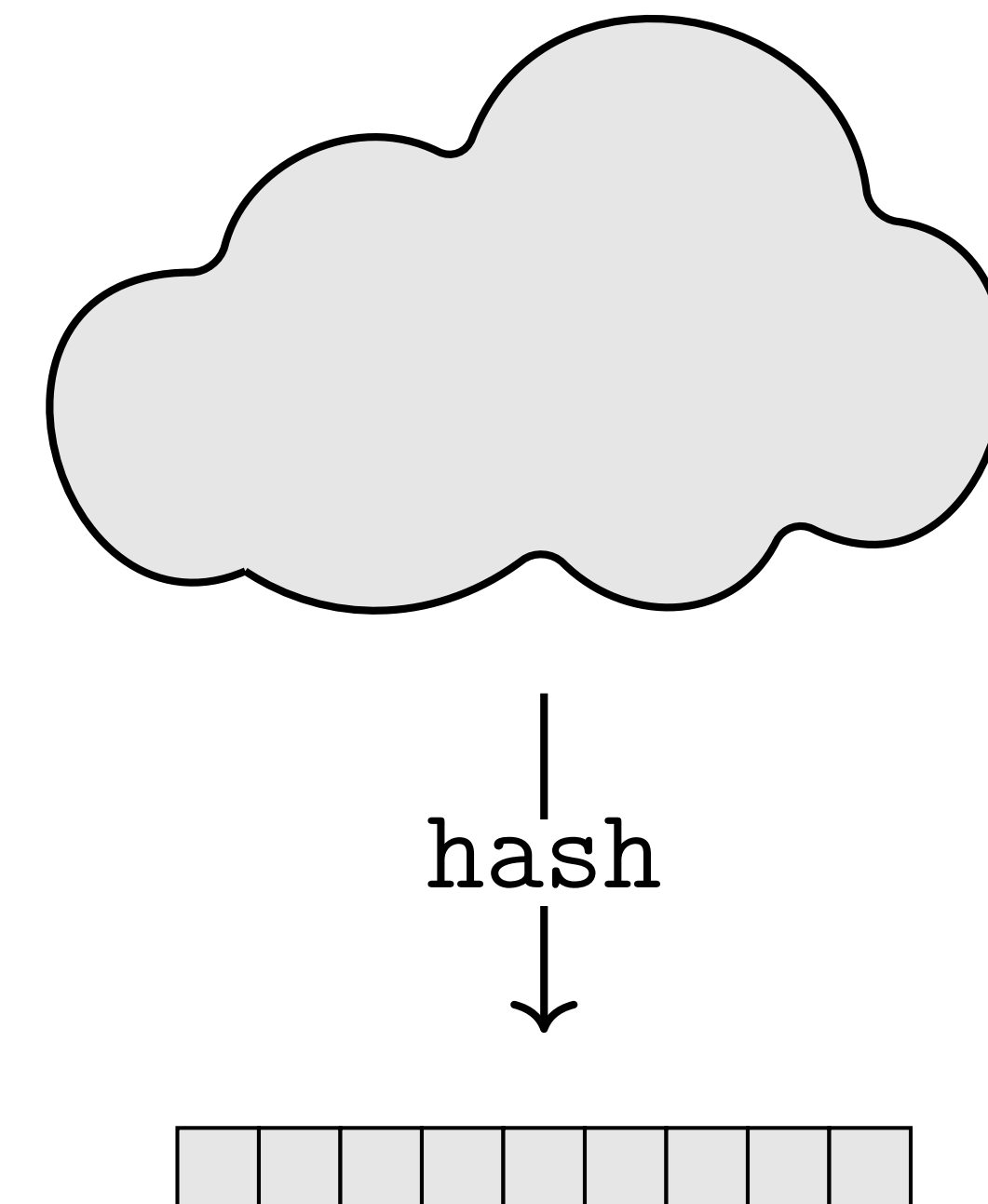
1. Determinismus - wenn $x = y$, dann ist auch $\text{hash}(x) = \text{hash}(y)$

2. „Kollisionsresistenz“ - eine gute Hashingfunktion minimiert die Wahrscheinlichkeit dass $\text{hash}(u) = \text{hash}(v)$ **wenn u ungleich v**

Hash-Tabelle

Genutzt, um ungeordnete Sammlungen einzigartiger Elemente zu speichern.

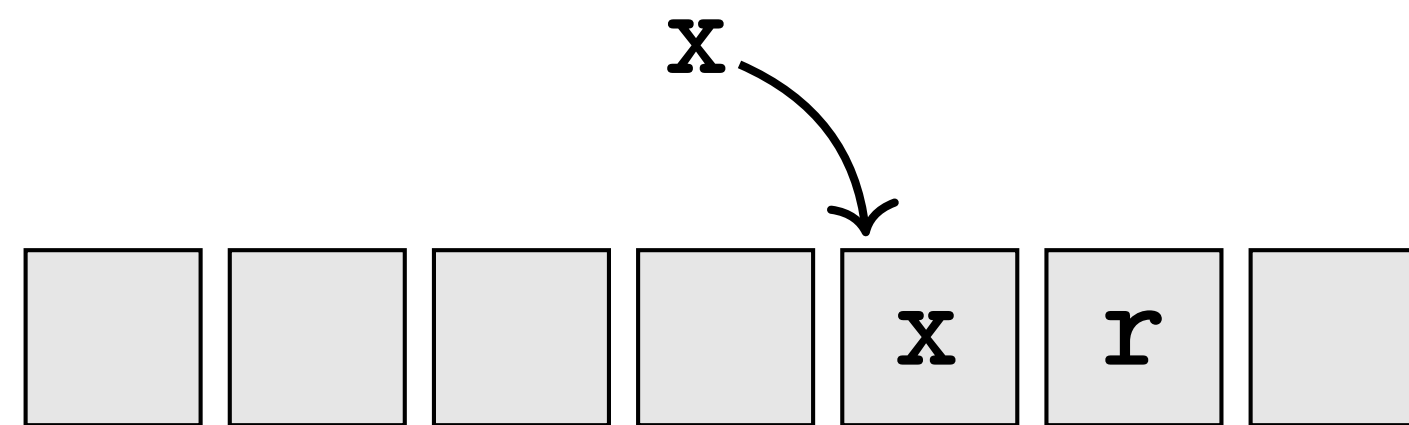
Operation	Erwartet	Schlechtester Fall
Suche	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Einfügen	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Entfernen	$\mathcal{O}(1)$	$\mathcal{O}(n)$



Kollisions-Behandlung

- Probing

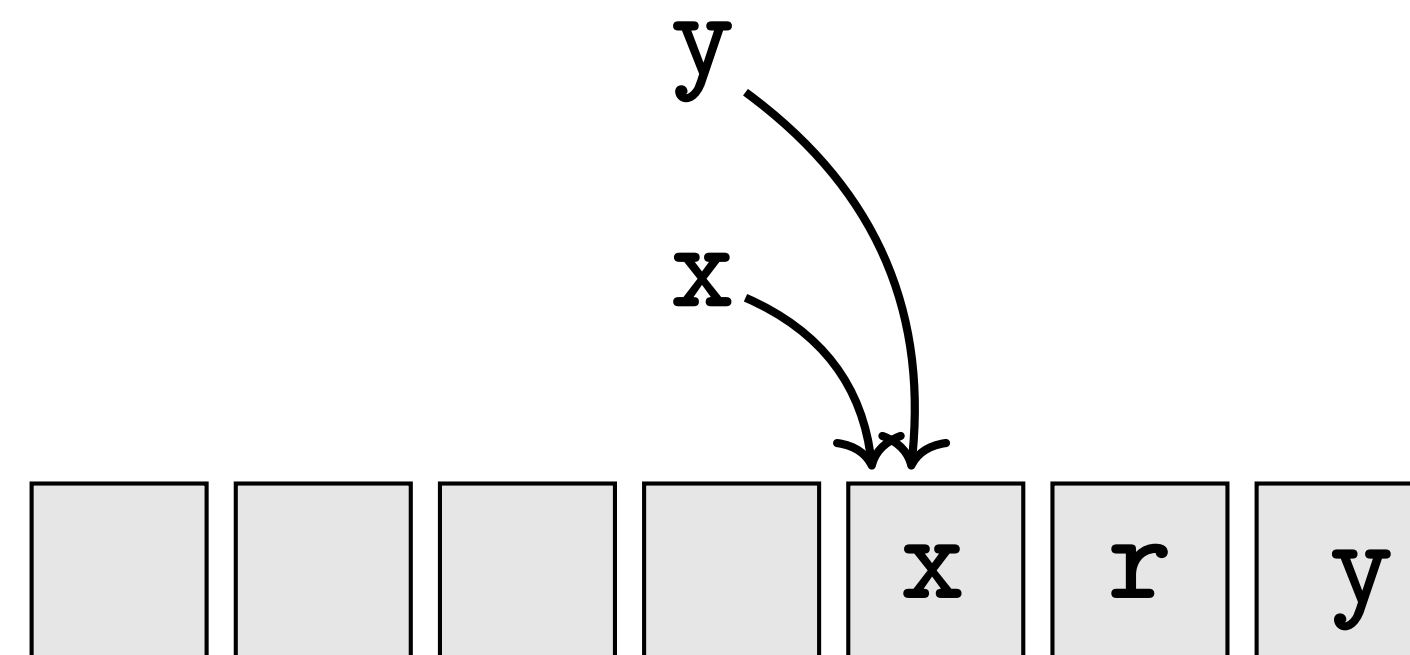
Wenn die Tabelle besetzt ist, wird der nächste freie Index gewählt



Kollisions-Behandlung

- Probing

Wenn die Tabelle besetzt ist, wird der nächste freie Index gewählt



wir weichen soweit nach rechts aus, bis es ein platz für y gibt.

Kollisions-Behandlung

- Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt



Kollisions-Behandlung

- Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt



Kollisions-Behandlung

- Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt

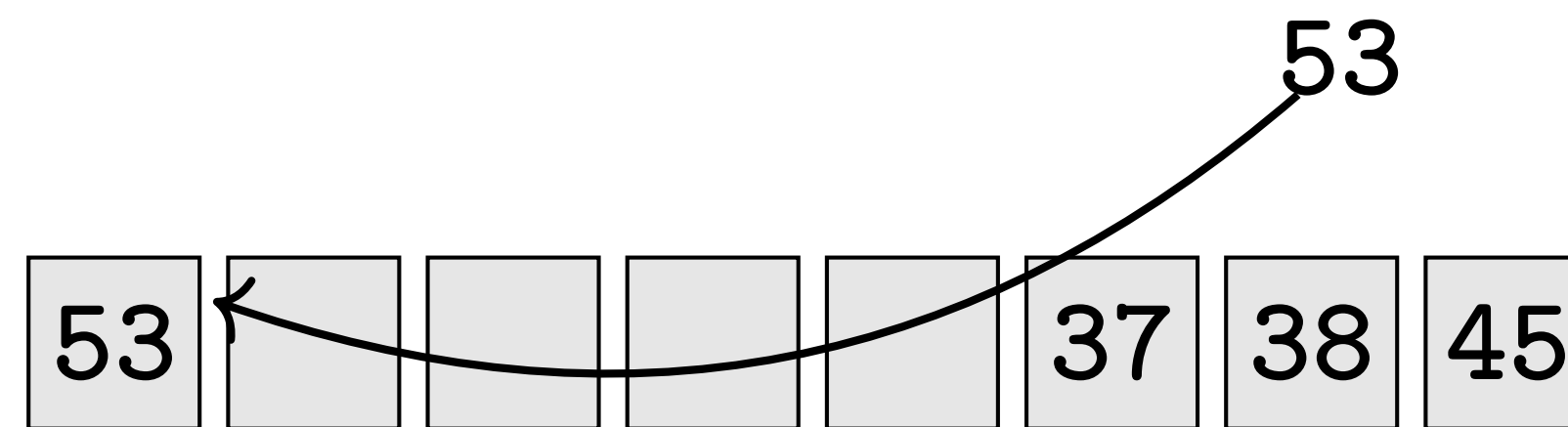


Kollisions-Behandlung

- Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt



Kollisions-Behandlung

- Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt

53					37	38	45
----	--	--	--	--	----	----	----

Wie viele Schritte braucht es, um 53 zu finden?

Kollisions-Behandlung

- Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt

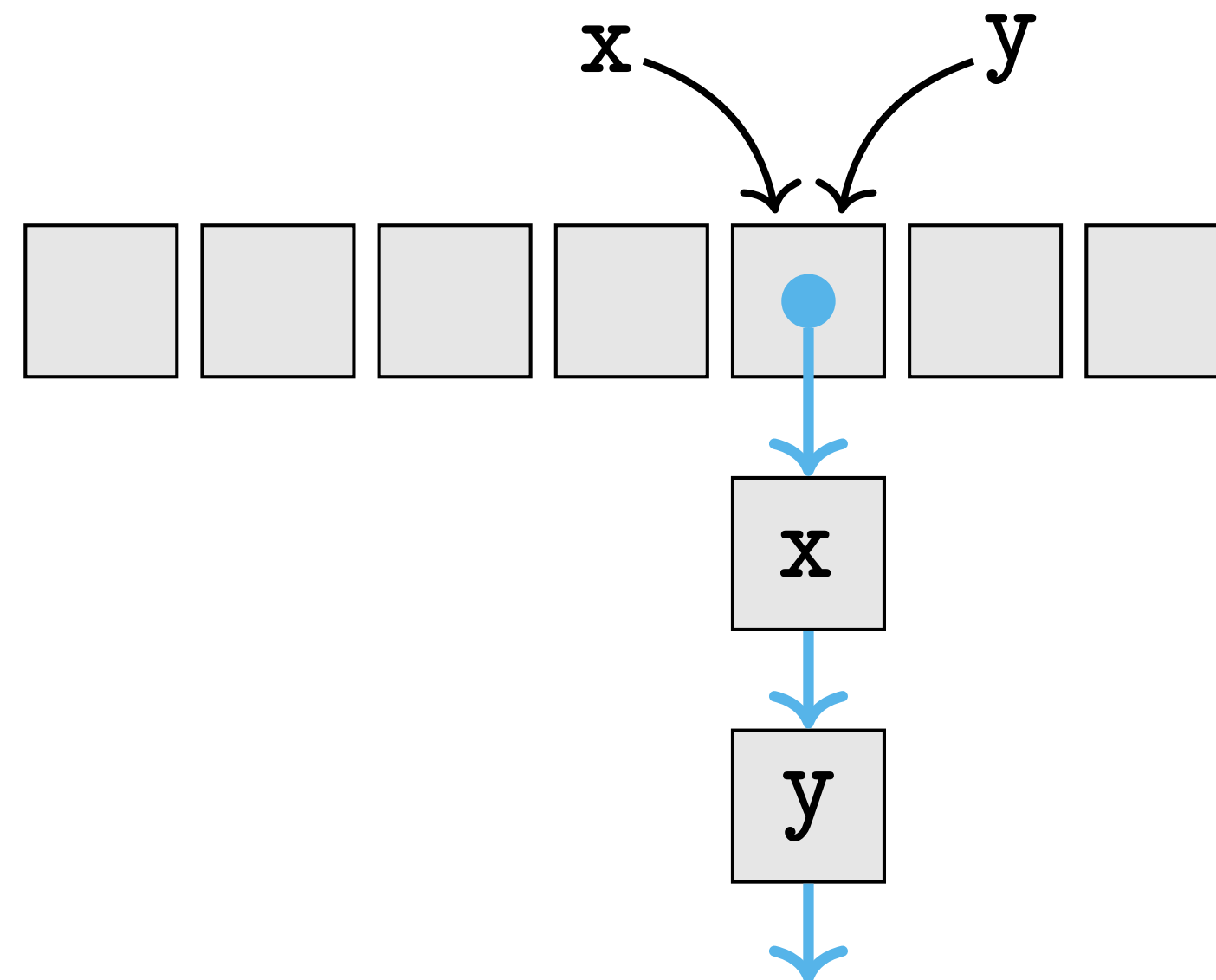


Wie viele Schritte braucht es, um 53 zu finden?

4Schritte, macht die schlechtester Fall laufzeit von $O(n)$ nun sinn?

Kollisions-Behandlung

- Chaining:
eine (verkettete) Liste von Elementen pro Tabellen-Eintrag



Kollisions-Behandlung

Probing und Chaining bieten beide gewisse Vorteile:

- Probing: Alle Daten bleiben im Array (Cache-freundlich)
- Chaining: Ein oft wiederholter Hash füllt nicht die ganze Hash-Tabelle, Zugriff auf andere Hashes bleibt schnell

Trotzdem haben beide Taktiken eine Worst-Case-Laufzeit von $\mathcal{O}(n)$.

klingt logisch, oder? - Top!

dict als Hash-Tabelle

keys	values
	
	
	

key wird gehashed $\text{hash}(\diamond)$

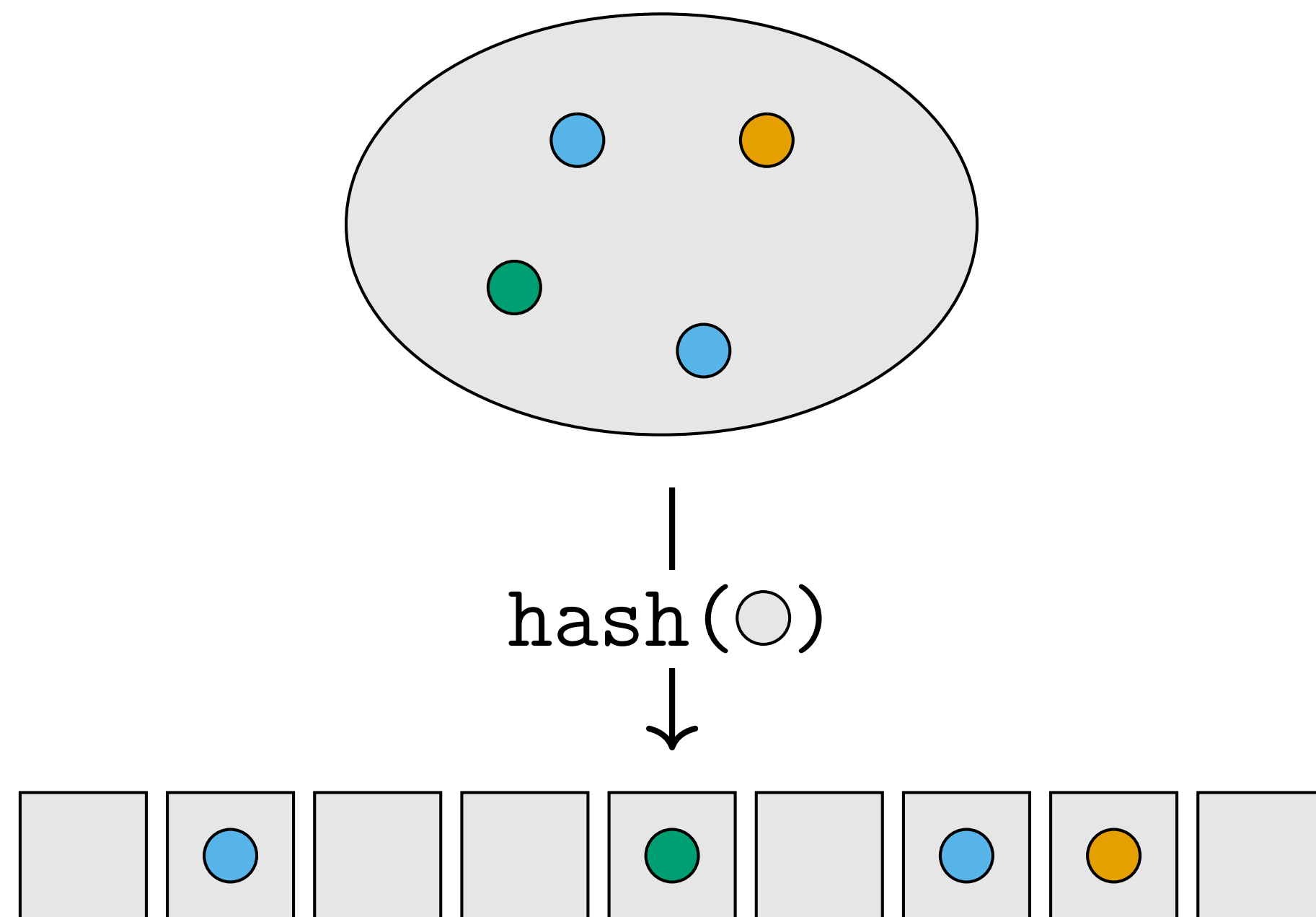


Operation	E	W
Suche	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Einfügen	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Entfernen	$\mathcal{O}(1)$	$\mathcal{O}(n)$

E : Erwartete Laufzeit

W : Laufzeit im Worst Case

set als Hash-Tabelle



Operation	E	W
Suche	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Einfügen	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Entfernen	$\mathcal{O}(1)$	$\mathcal{O}(n)$

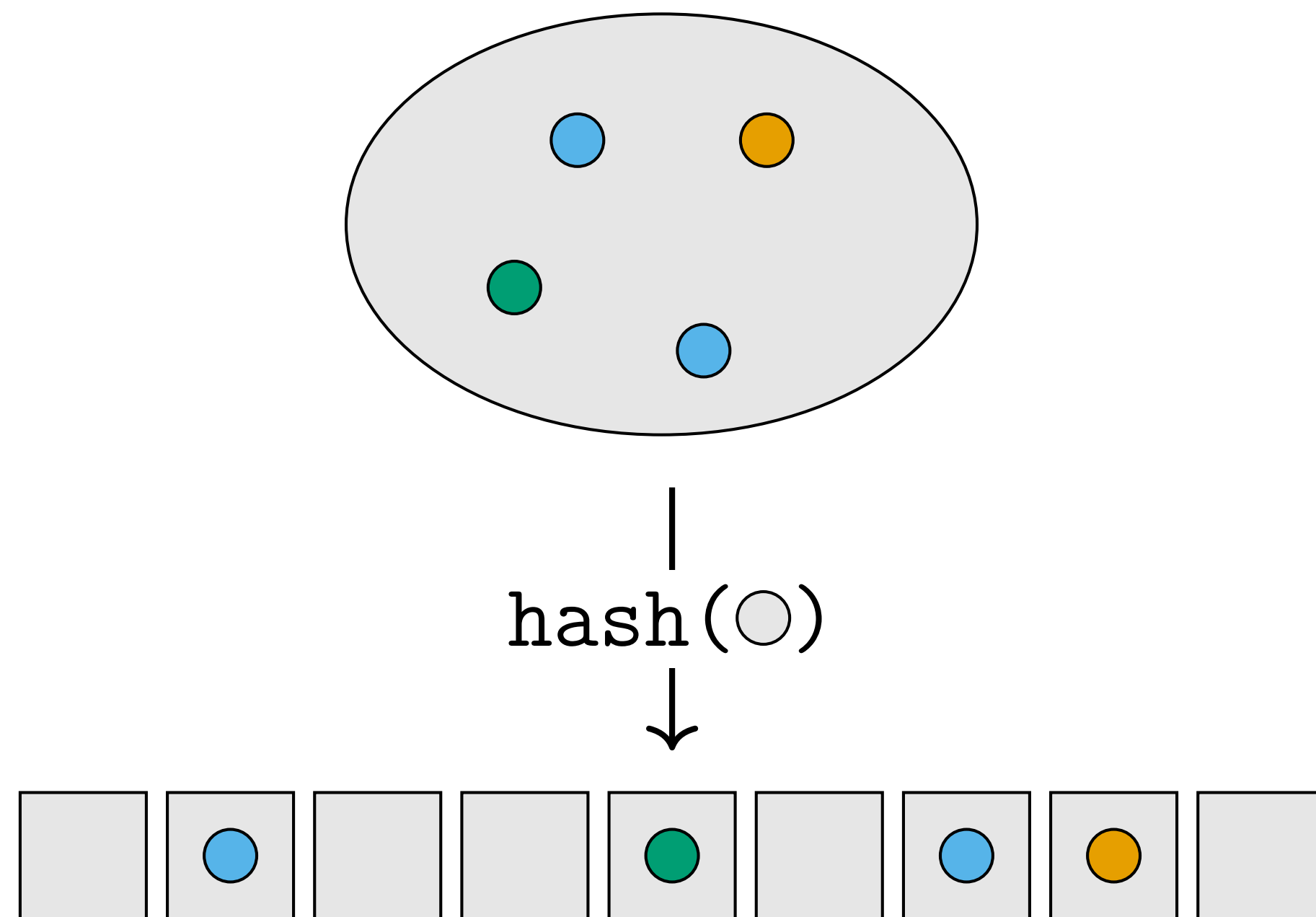
E : Erwartete Laufzeit

W : Laufzeit im Worst Case

set ist eigentlich wie ein Dictionary, welches nur Keys enthält

set als Hash-Tabelle

set ist eigentlich wie ein Dictionary, welches nur Keys enthält



Operation	E	W
Suche	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Einfügen	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Entfernen	$\mathcal{O}(1)$	$\mathcal{O}(n)$

E : Erwartete Laufzeit

W : Laufzeit im Worst Case

Frage fürs Bigger picture: Was ist der Sinn einer schnellen suche wenn ich das element ja „bereits kennen muss“

Kollisions-Behandlung

■ Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt



Wie viele Schritte braucht es, um 53 zu finden? 4 Schritte ausgehend von Index 5

Wie viele Schritte braucht es, um sicherzustellen dass 61 NICHT enthalten ist?

Tipp: $61 \bmod 8 = 5 = 37 \bmod 8$

Kollisions-Behandlung

■ Probing

Beispiel: Die Hash-Funktion ist $H(x) = (x \bmod 8)$

Wenn Tabelle besetzt ist, wird der nächste freie Index gewählt



Wie viele Schritte braucht es, um 53 zu finden? 4 Schritte ausgehend von Index 5

Wie viele Schritte braucht es, um sicherzustellen dass 61 NICHT enthalten ist?

5 Schritte, beginnend bei Index 5 bis ein leerer Knoten erreicht wird

die leeren Plätze versichern uns, dass die 61 nicht aufgrund von kollisionen¹¹ doch enthalten ist

Set/Dict als BBST?

- Könnte man ein Set auch als balancierten binären Suchbaum implementieren?
Falls ja, wie und mit welcher Laufzeit? Falls nein, weshalb nicht?

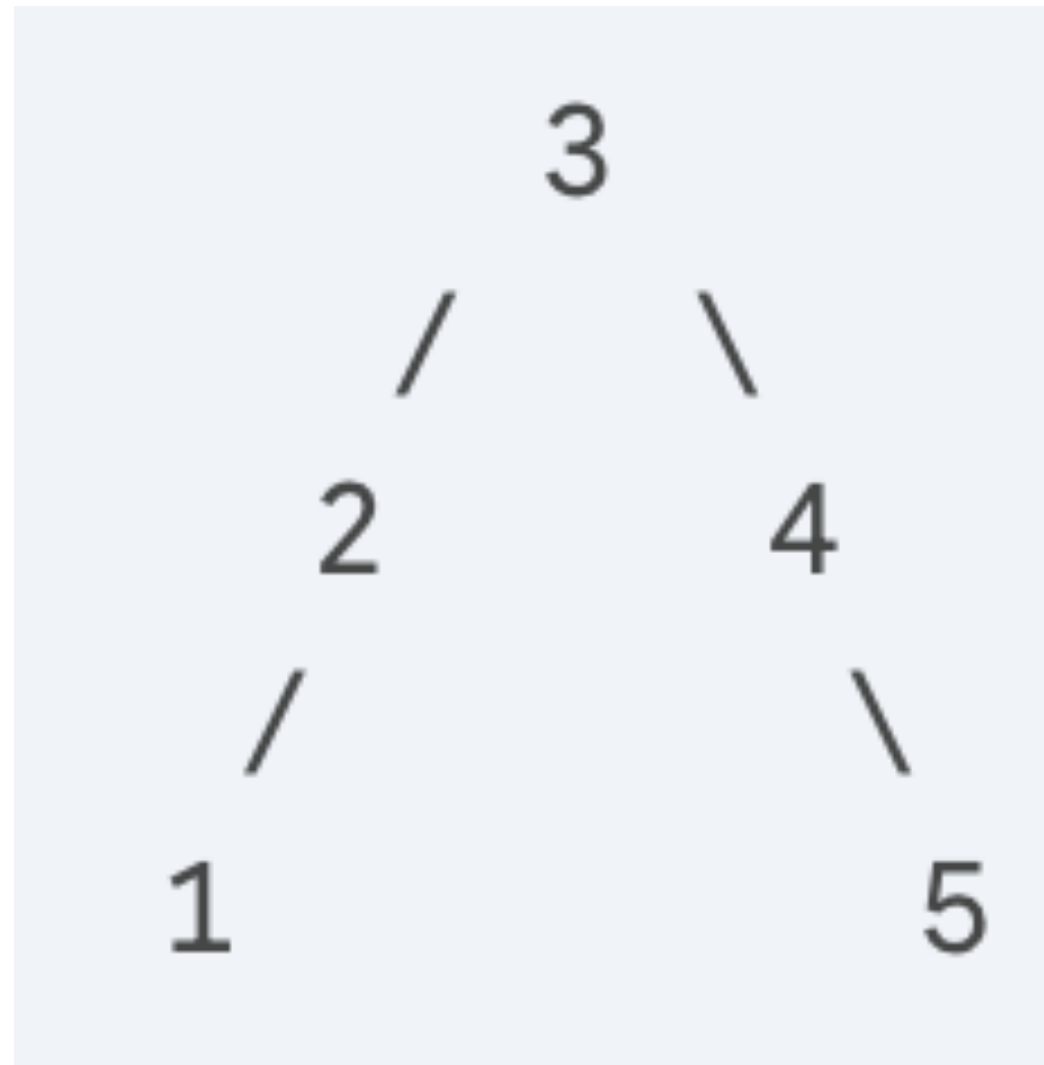
übersetzt: Könnte man für ein Set auch ein BBST benutzen anstatt ein Hash table

Set/Dict als BBST?

- Könnte man ein Set auch als balancierten binären Suchbaum implementieren?
Falls ja, wie und mit welcher Laufzeit? Falls nein, weshalb nicht?
Ja! Jedes Element in der Menge ist ein Knoten im Suchbaum. $\mathcal{O}(\log n)$ für Einfügen/Entfernen/Suchen. Besser worst-case-Garantie als Hash-Tabellen.

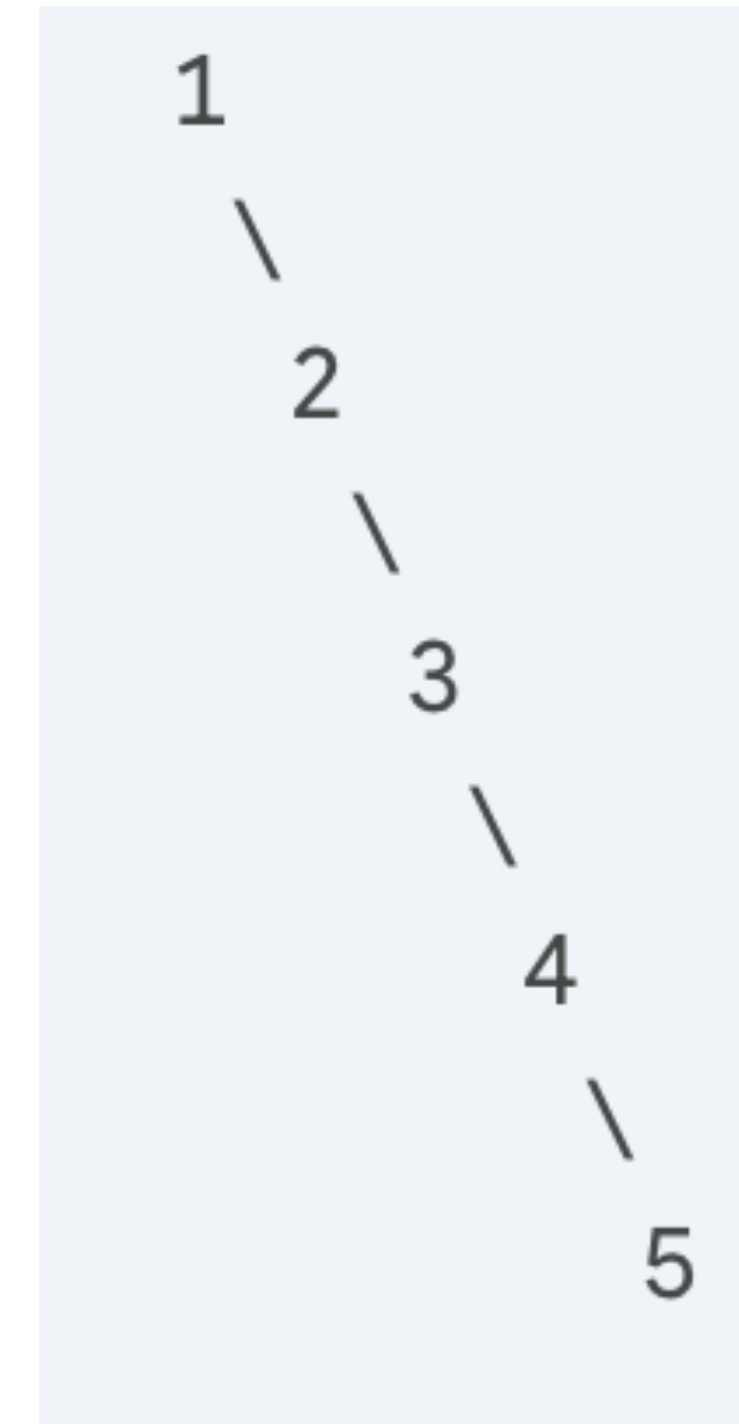
weshalb muss der BST balanciert sein?

weshalb muss der BST balanciert sein?



3 schritte um die 5 zu finden

hinweis: $\log(n=5) = 2.32$, aufgerundet 3.



5 schritte um die 5 zu finden.

n=5

Könnte man dementsprechend auch ein Dictionary als BST implementieren?

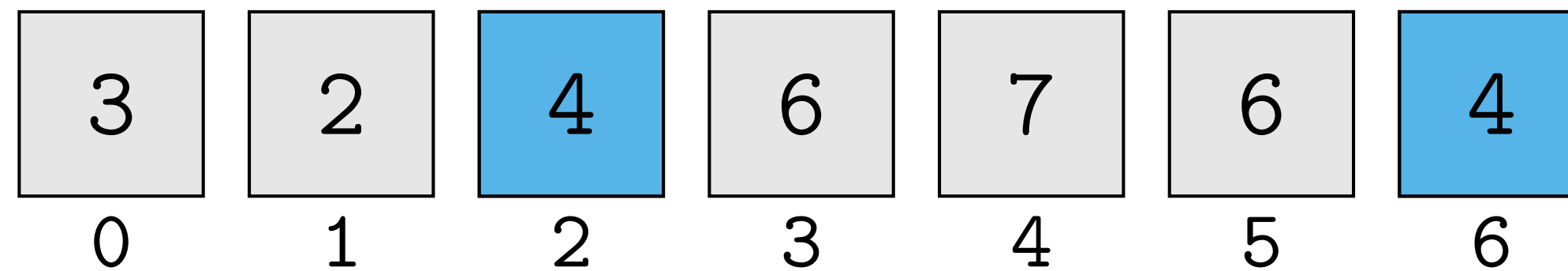
Könnte man dementsprechend auch ein Dictionary als BST implementieren?

absolut, hier speichern wir in einem Knoten einfach ein tupel (key,value)
Der Baum ist nach **key sortiert!**

Duplikate entfernen mit einer Liste

Input: eine Liste `a`

Output: `True` falls ein Element mehrmals in `a` vorkommt, `False` sonst

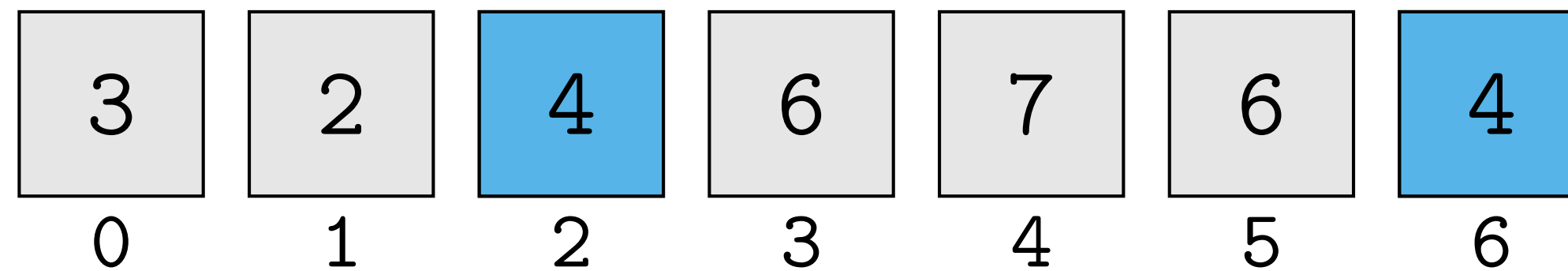


überlegen: (Pseudo-)Code um zu prüfen, ob ein Element mehrfachvorkommt

Duplikate entfernen mit einer Liste

Input: eine Liste `a`

Output: `True` falls ein Element mehrmals in `a` vorkommt, `False` sonst



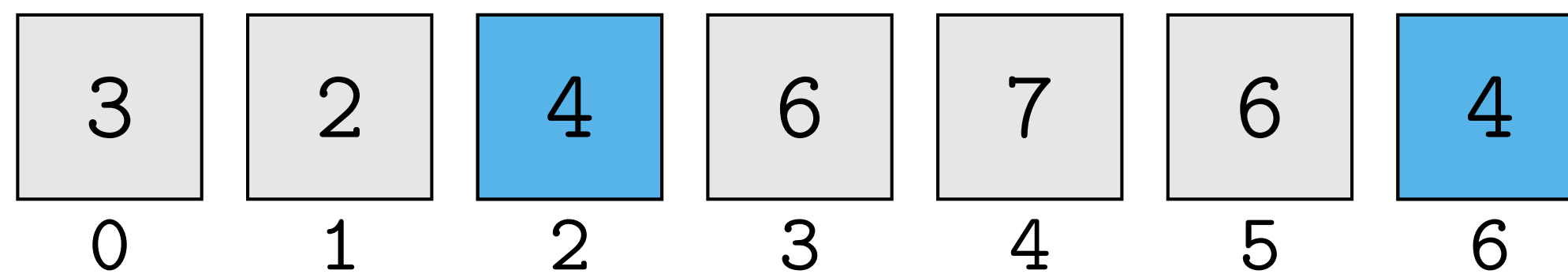
```
1 def has_duplicates(a):
2     """Return True if a has duplicates."""
3     for index, x in enumerate(a):
4         if x in a[:index]:
5             return True
6     return False
```

O Laufzeit davon?

Duplikate entfernen mit einer Liste

Input: eine Liste `a`

Output: `True` falls ein Element mehrmals in `a` vorkommt, `False` sonst



```
1 def has_duplicates(a):
2     """Return True if a has duplicates."""
3     for index, x in enumerate(a):
4         if x in a[:index]:           x in liste , dauert len(liste) im worst case.
5         return True
6     return False
```

Laufzeit für $n = \text{len}(a)$: $\leq \sum_{i=1}^n \mathcal{O}(i) \in \mathcal{O}(n^2)$

übrigens, **schnellere alternative** ($\mathcal{O}(n)$): `return len(liste)==len(set(liste))`

*nur zum prüfen nützlich, wir haben keinen index um zu entfernen danach

Duplikate entfernen mit einem Set

```
1 def has_duplicates(a):
2     """Return True if a has duplicates."""
3     seen_elements = set()
4     for x in a:
5         if x in seen_elements:
6             return True
7         else:
8             seen_elements.add(x)
9     return False
```

Laufzeit für $n = \text{len}(a)$:

finde erwartete und worst-case laufzeit

tipp: wodurch könnte hier der worst-case entstehen

Duplikate entfernen mit einem Set

```
1 def has_duplicates(a):  
2     """Return True if a has duplicates."""  
3     seen_elements = set()  
4     for x in a:  
5         if x in seen_elements:  
6             return True  
7         else:  
8             seen_elements.add(x)  
9     return False
```

hat erwartet $O(1)$

Laufzeit für $n = \text{len}(a)$:

erwartet: $\leq \sum_{i=1}^n c \in \mathcal{O}(n)$

Worst Case: $\leq \sum_{i=1}^n c \cdot i \in \mathcal{O}(n^2)$ aufgrund von Kollisionen!

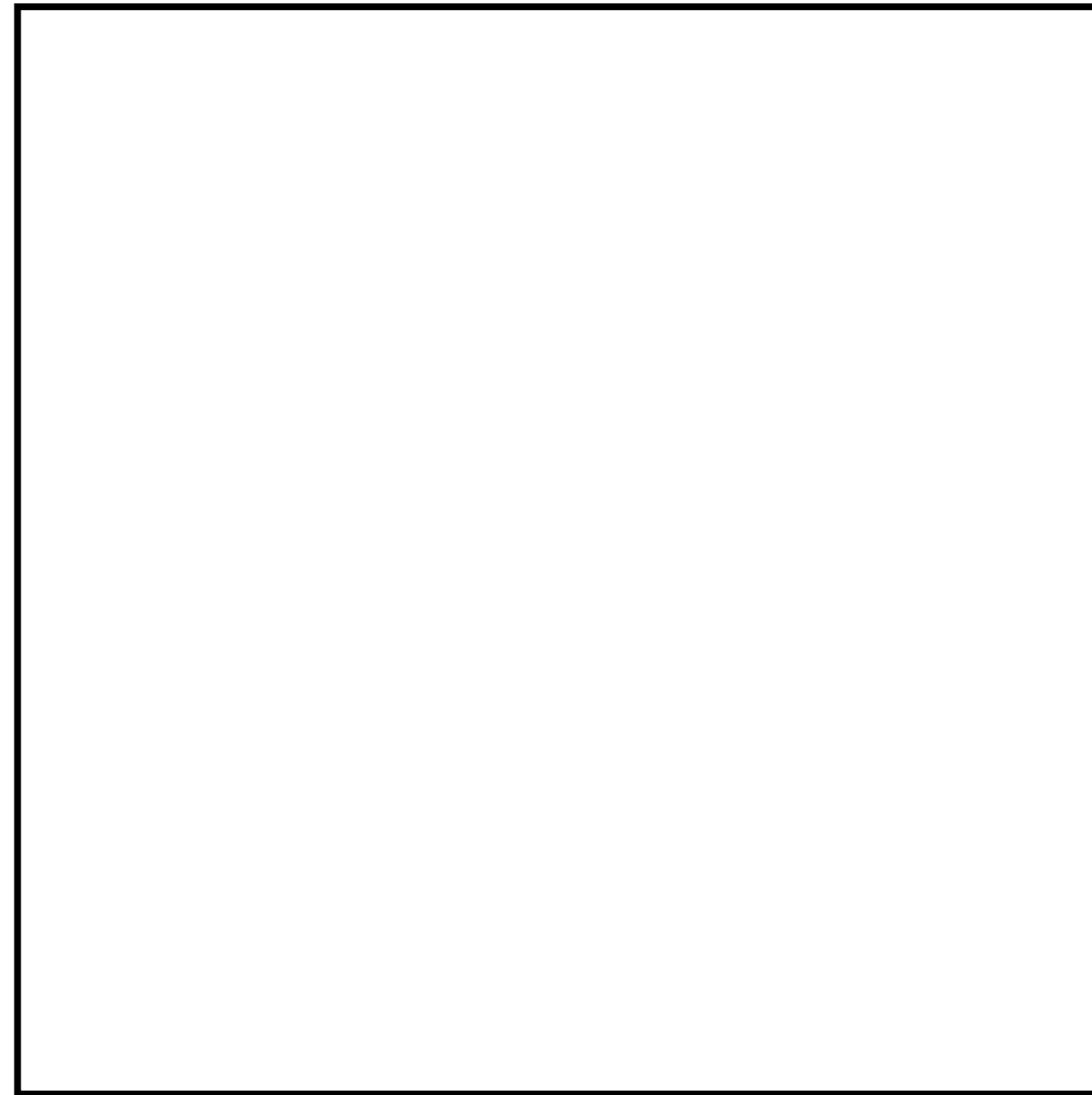
Quadtrees

Quadrees Konzept

ein **Viereck**, mit räumlichen Datenpunkten innerhalb des Vierecks

bsp. Google Maps

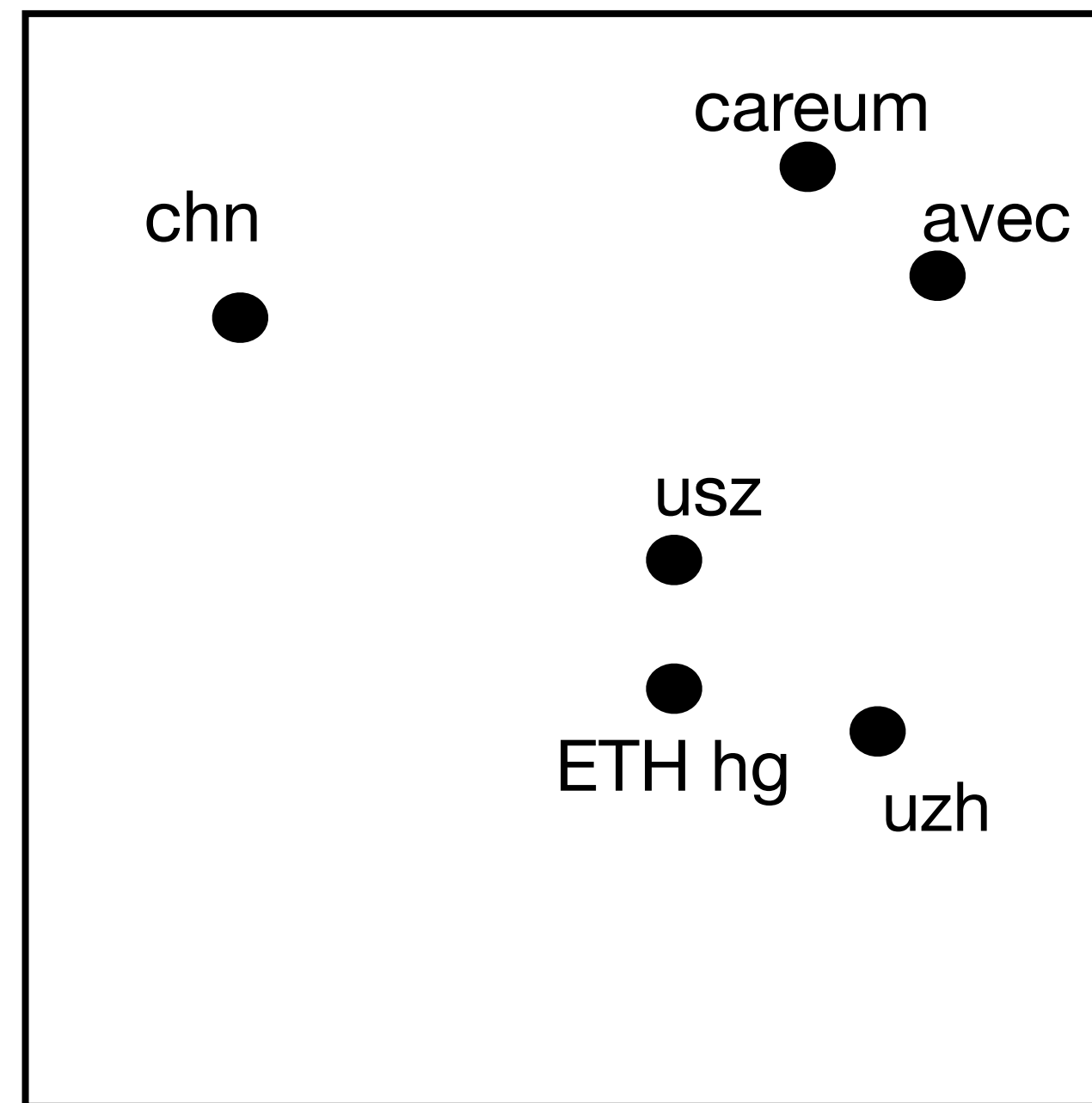
Handy Bildschirm =>



Quadrees Konzept

ein **Viereck**, mit räumlichen Datenpunkten innerhalb des Vierecks

bsp. campus Map



jeder dieser punkte hat eine
Koordinate / fixe Position
innerhalb des Viereckes

Handy Bildschirm =>
(Quadtree)

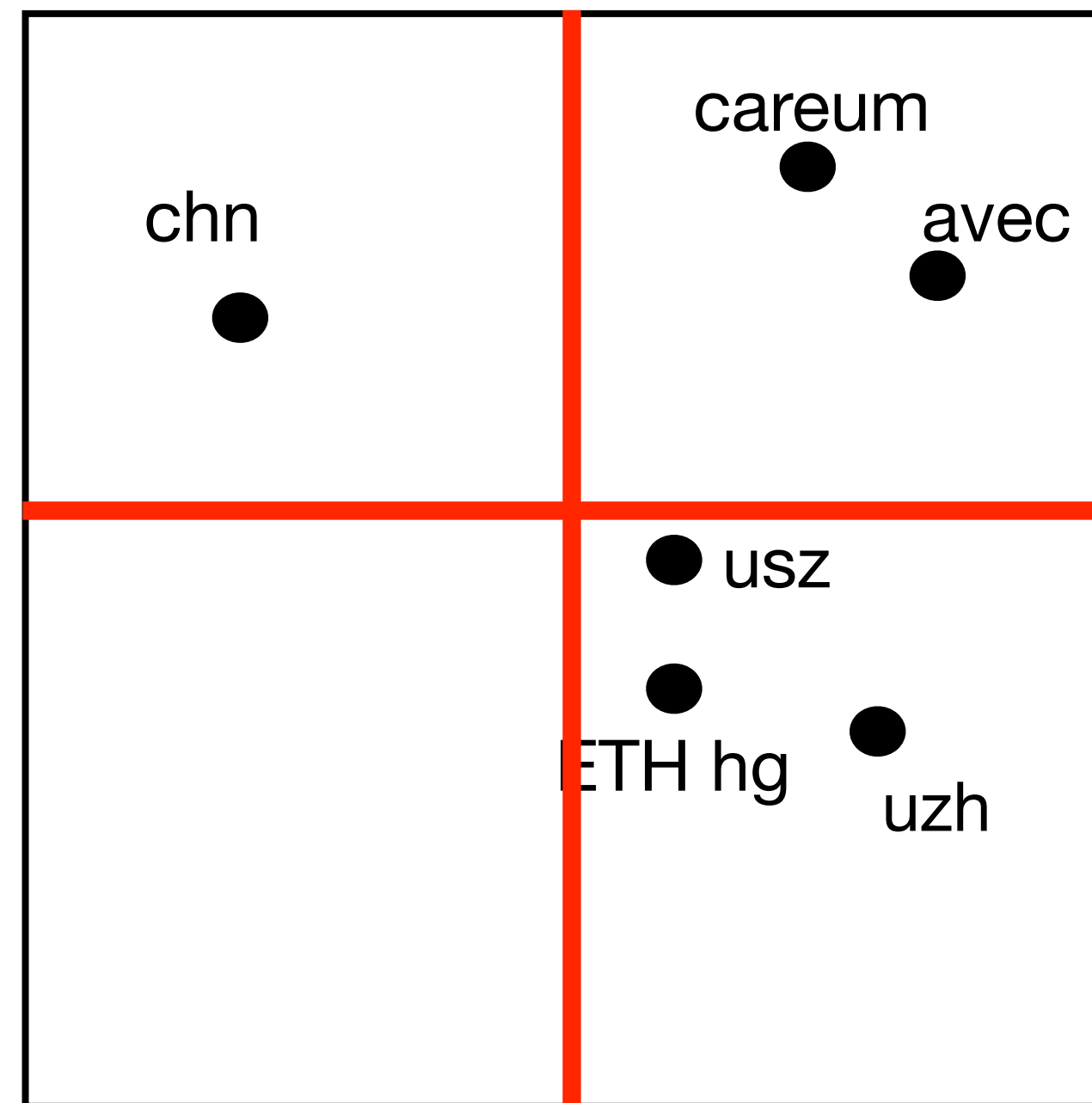
Quadtrees Konzept

ein **Viereck**, mit räumlichen Datenpunkten innerhalb des Vierecks

bsp. campus Map

unser QuadTree
nimmt **max 3**
Elemente auf (pro
Parzelle),

bei einer
Überschreitung **teilen**
wir in 4

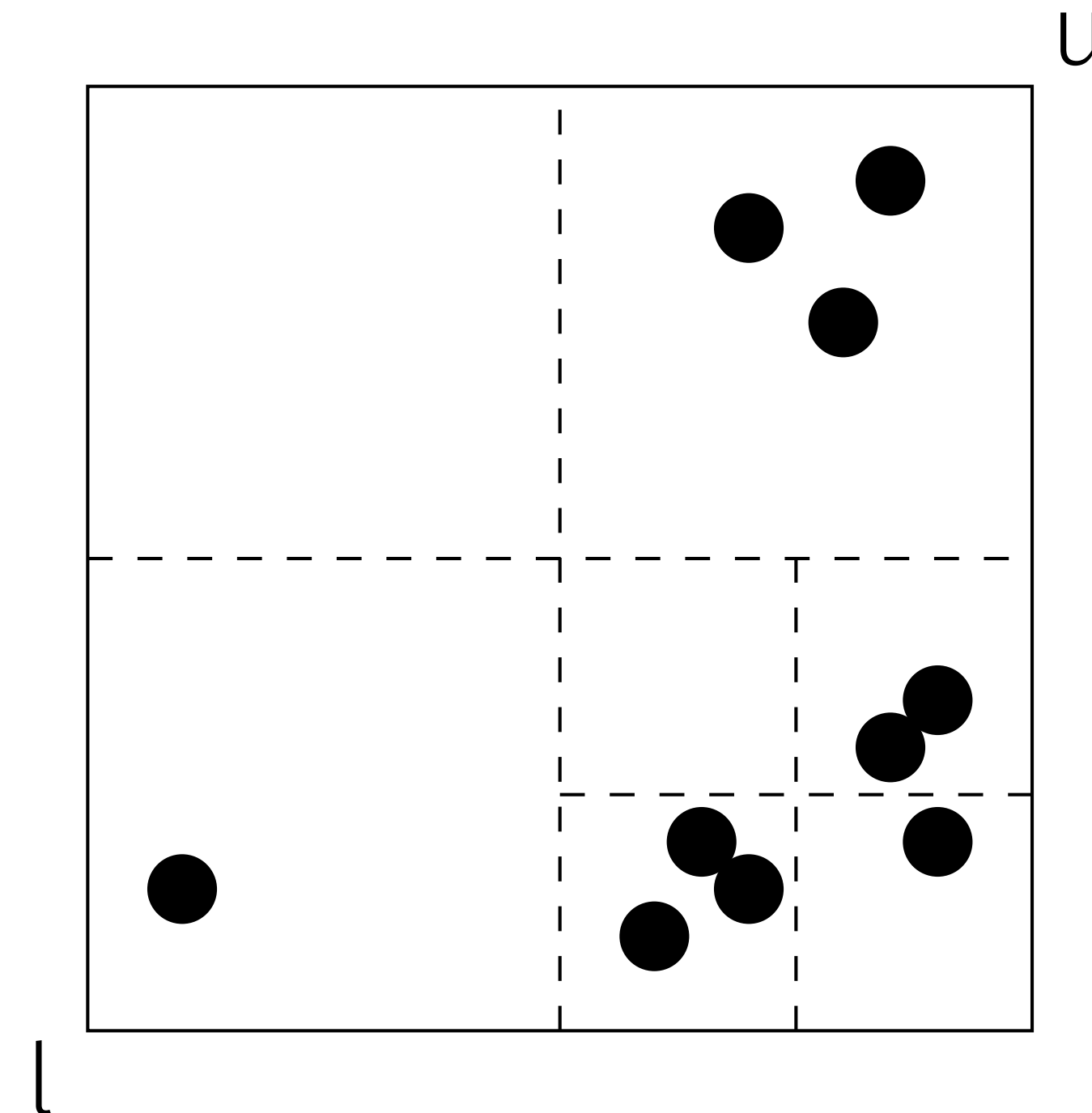


jeder dieser punkte hat eine Koordinate / fixe Position innerhalb des Vierecks

Idee

Das Hinzufügen weiterer Punkte kann zu einer weiteren Unterteilung der Quadrees führen.

Ein Quadtree mit $m = 3$.



Beachte, wie „lokal, geteilt wird“ unten rechts eher „hoch aufgelöst“ in allen anderen „grob aufgelöst“

Vorteile Quadrees

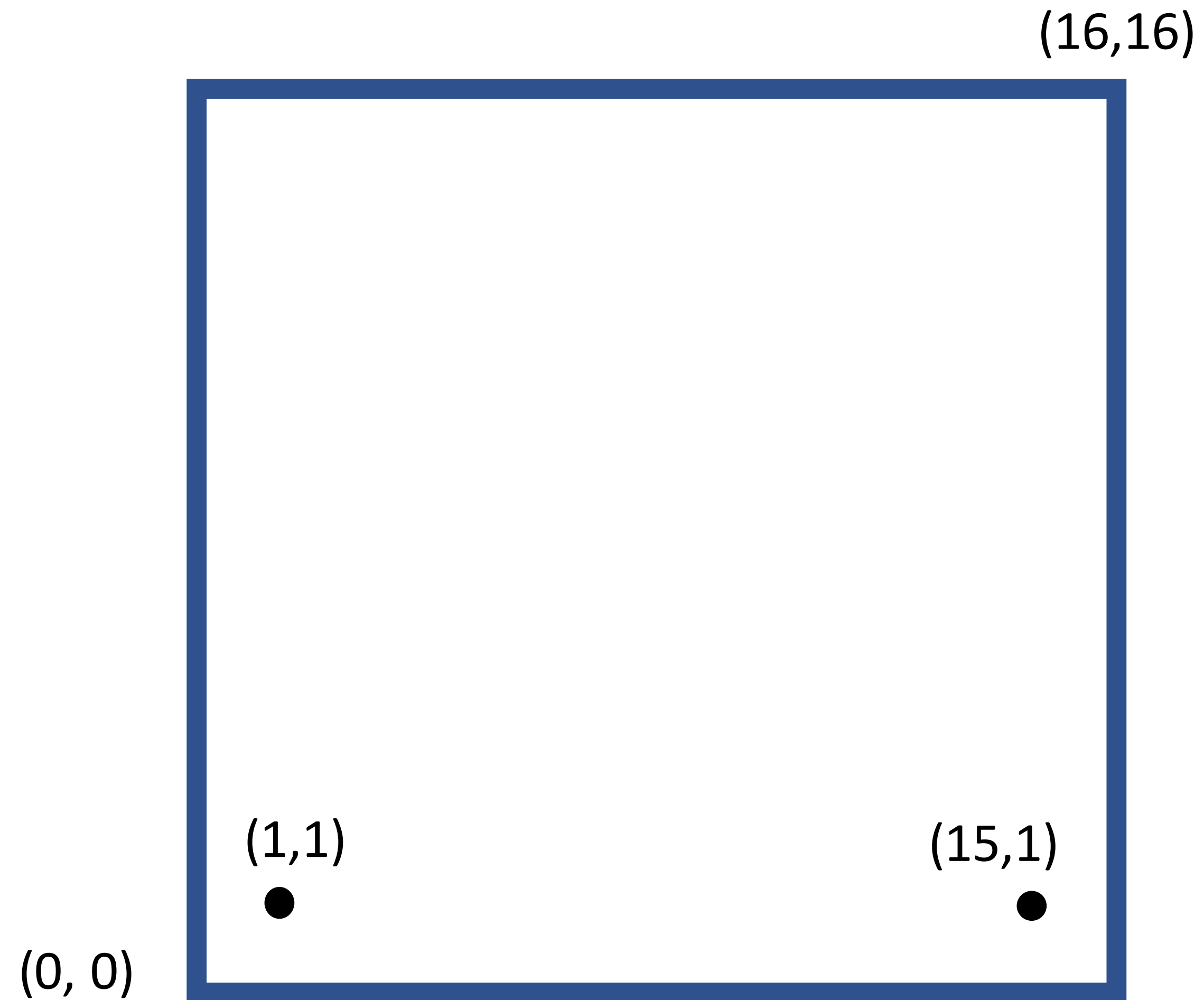
- Dynamische Auflösung
- Suchezeit von $\text{Log}(n)$ (klärt sich noch)

Lösung: Erstellen eines Quadtree

Max capacity : 2

Points to insert:

(1,1)
(15,1)
(5,2)
(5,5)
(7,7)
(7,5)
(10,10)
(9,15)
(15,14)

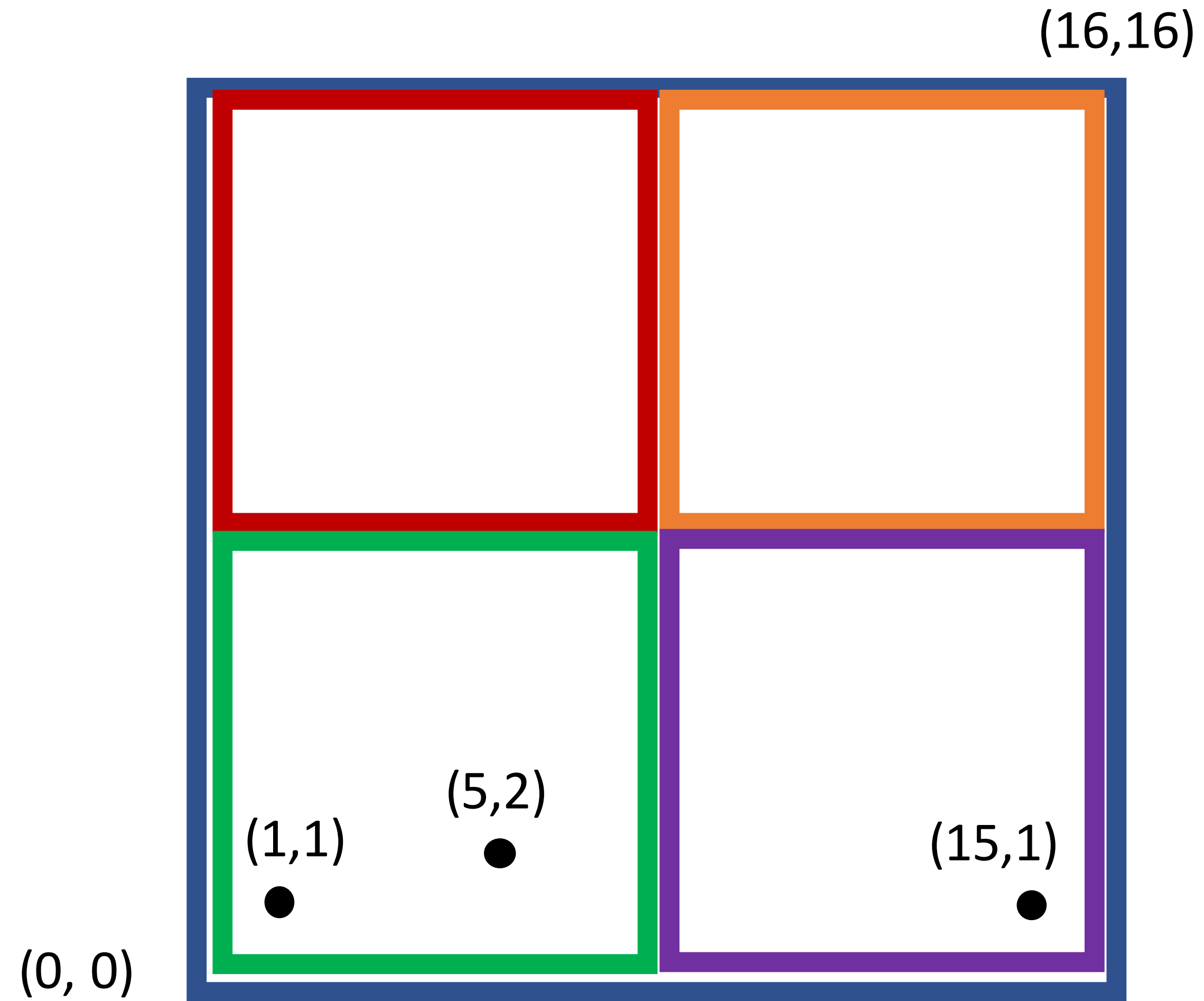


Lösung: Erstellen eines Quadtree

Max capacity : 2

Points to insert:

(1,1)
(15,1)
(5,2)
(5,5)
(7,7)
(7,5)
(10,10)
(9,15)
(15,14)

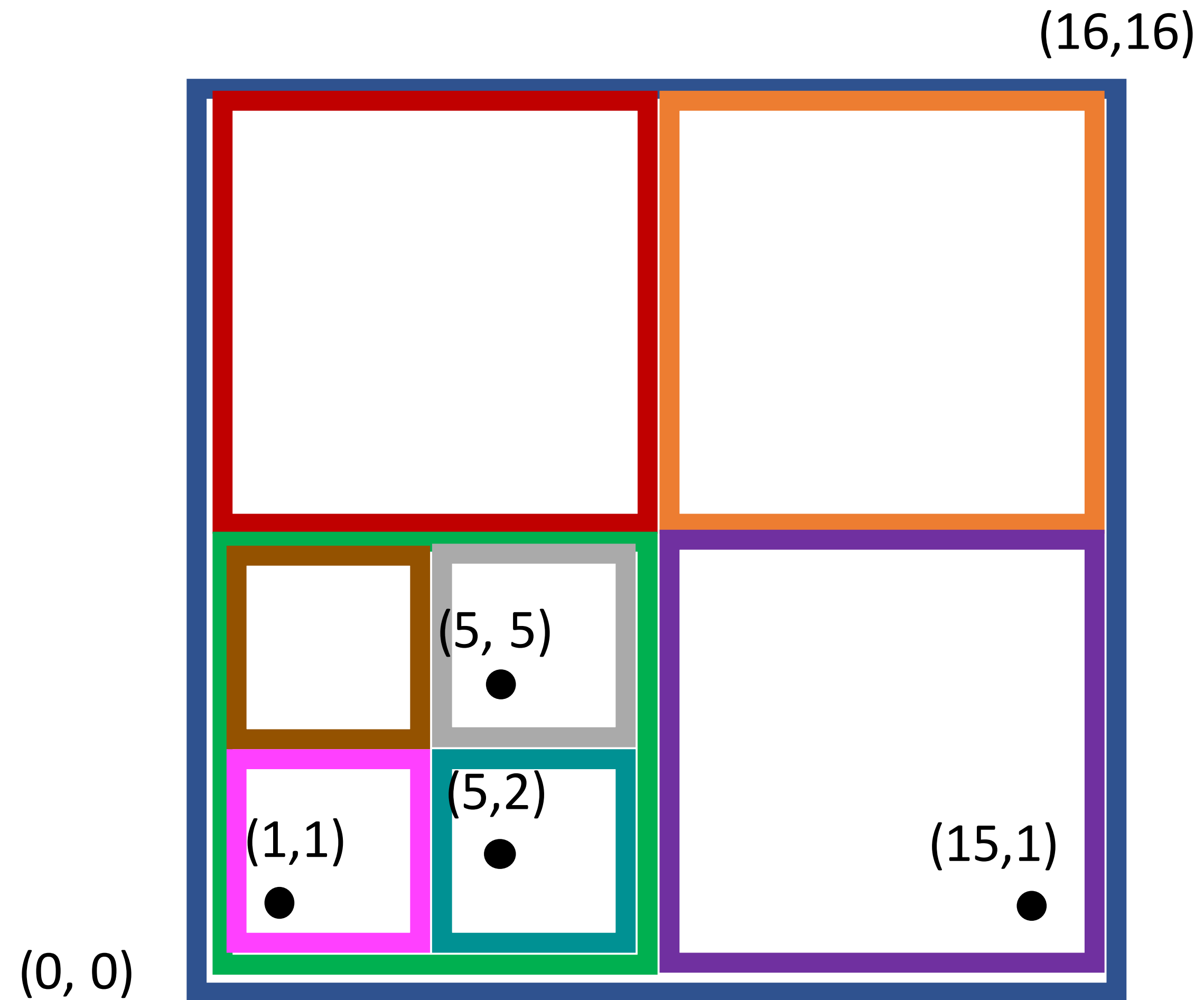


Lösung: Erstellen eines Quadtree

Max capacity : 2

Points to insert:

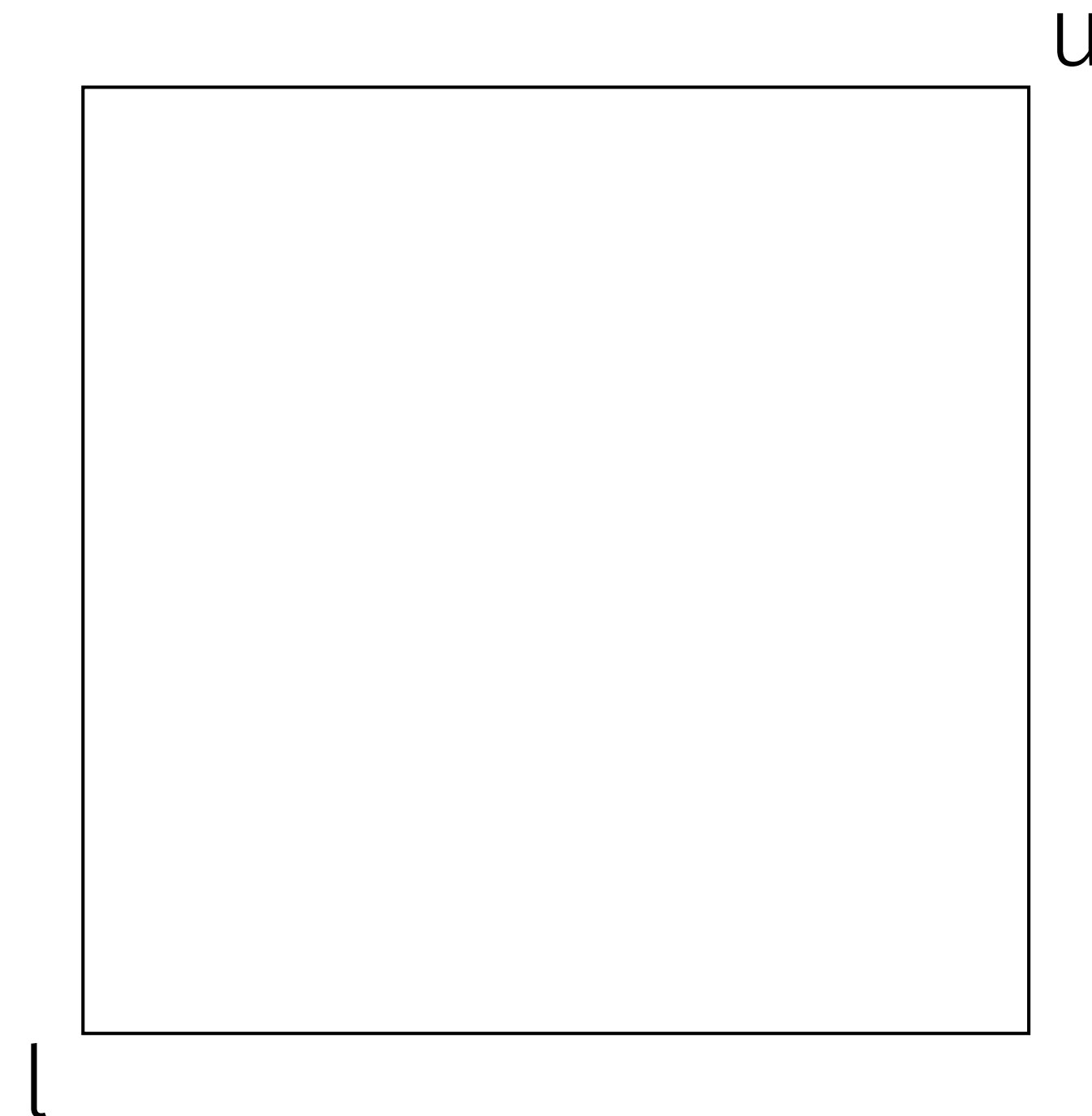
(1,1)
(15,1)
(5,2)
(5,5)
(7,7)
(7,5)
(10,10)
(9,15)
(15,14)



Idee

```
1 class QuadTree:
2     def __init__(self, l, u,
3         max_cap):
4         self.l = l
5         self.u = u
6         self.m = max_cap
7         self.points = []
8         self.children = None
9         self.count = 0
```

Ein Quadtree ist leer am Anfang.

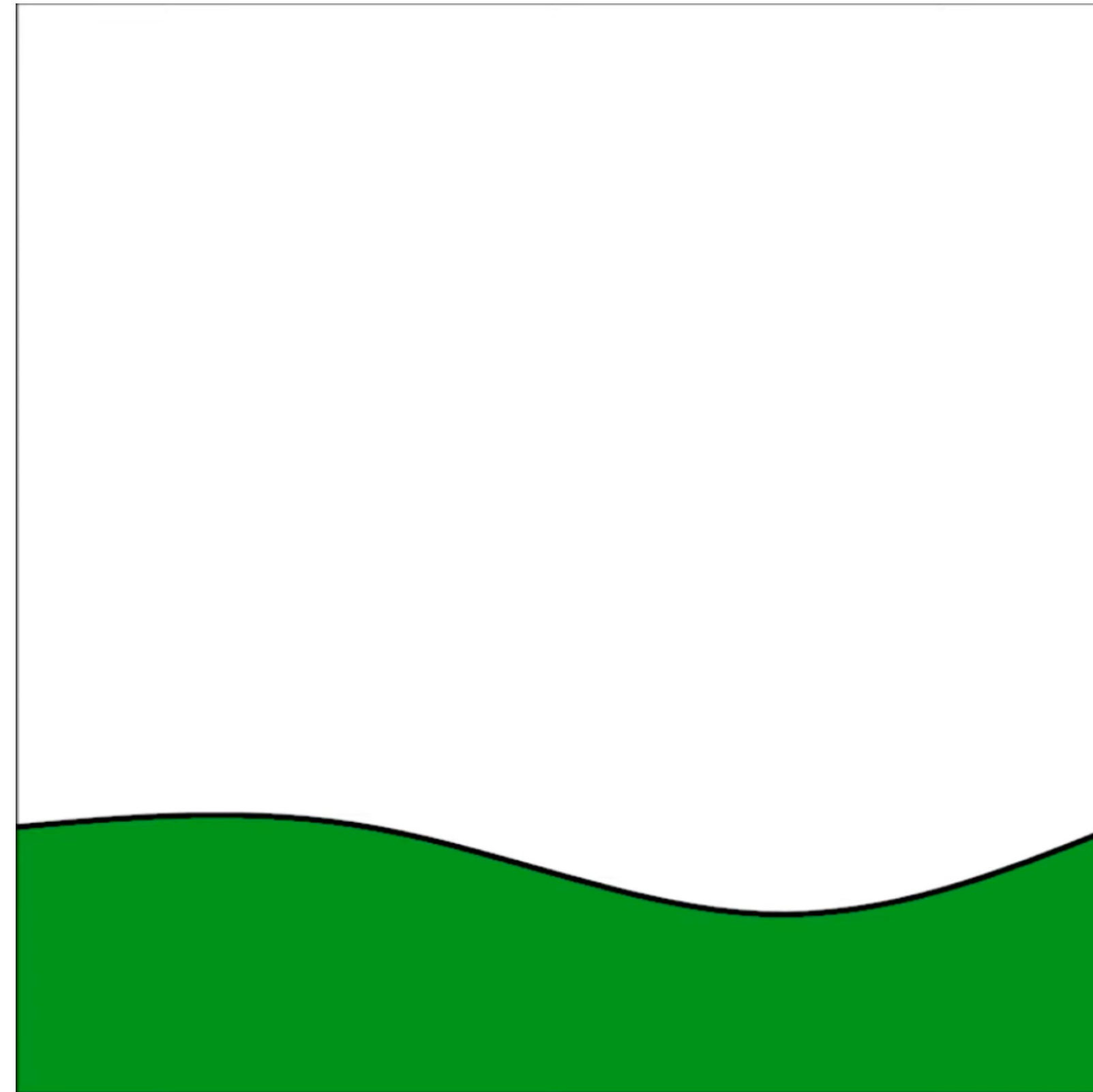


Vorstellen Wie ein **Baum** mit 4 kindern!

Beispiel

Das ganze Konzept lässt sich hier raus ableiten!

Ziel: diese „landschaft“
mittels Quadtree
beschreiben

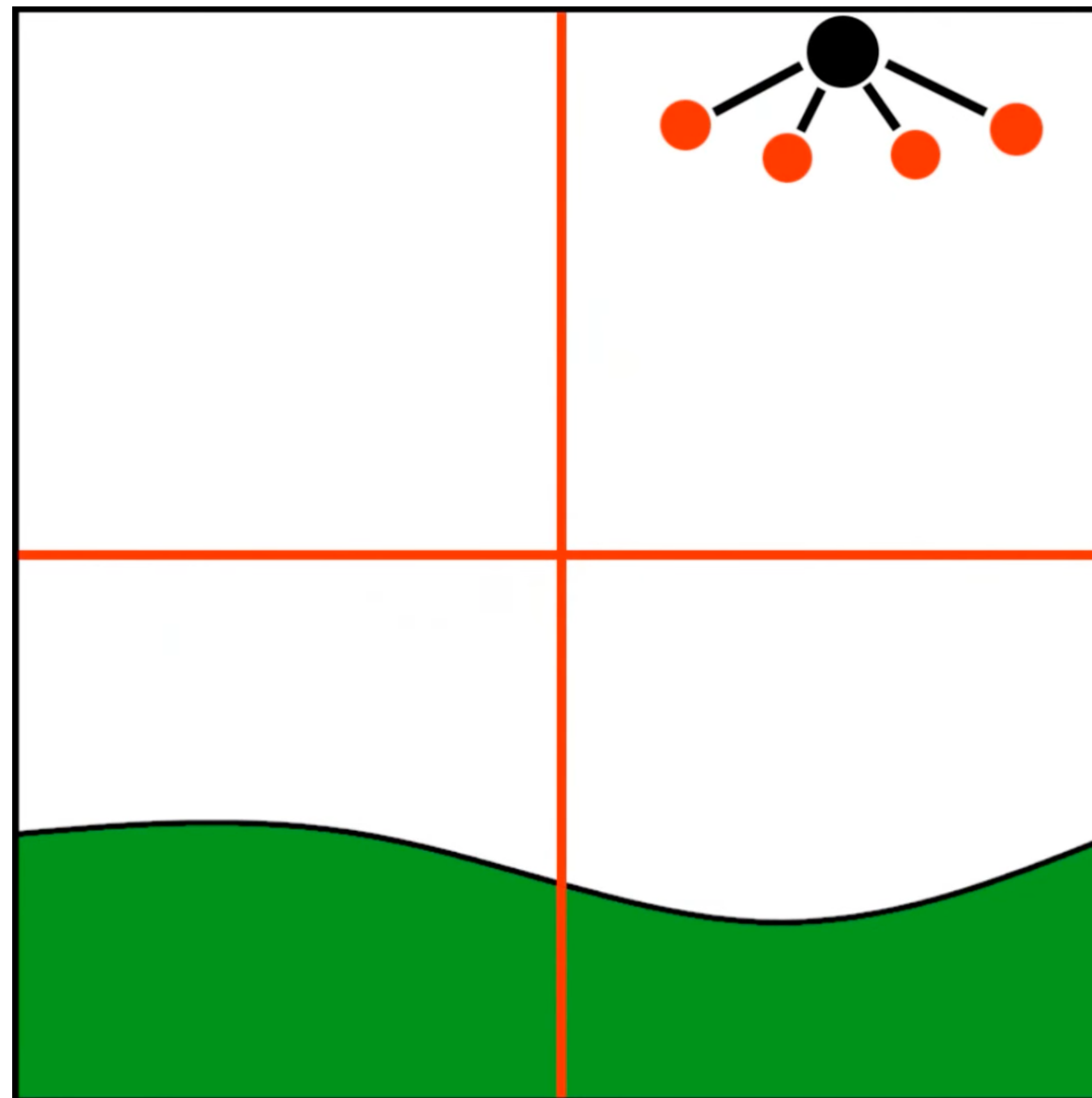


Beispiel

Das ganze Konzept lässt sich hier raus ableiten!

Der **Baum**, wie wir dies speichern könnten

Ziel: diese „landschaft“
mittels Quadtree
beschreiben

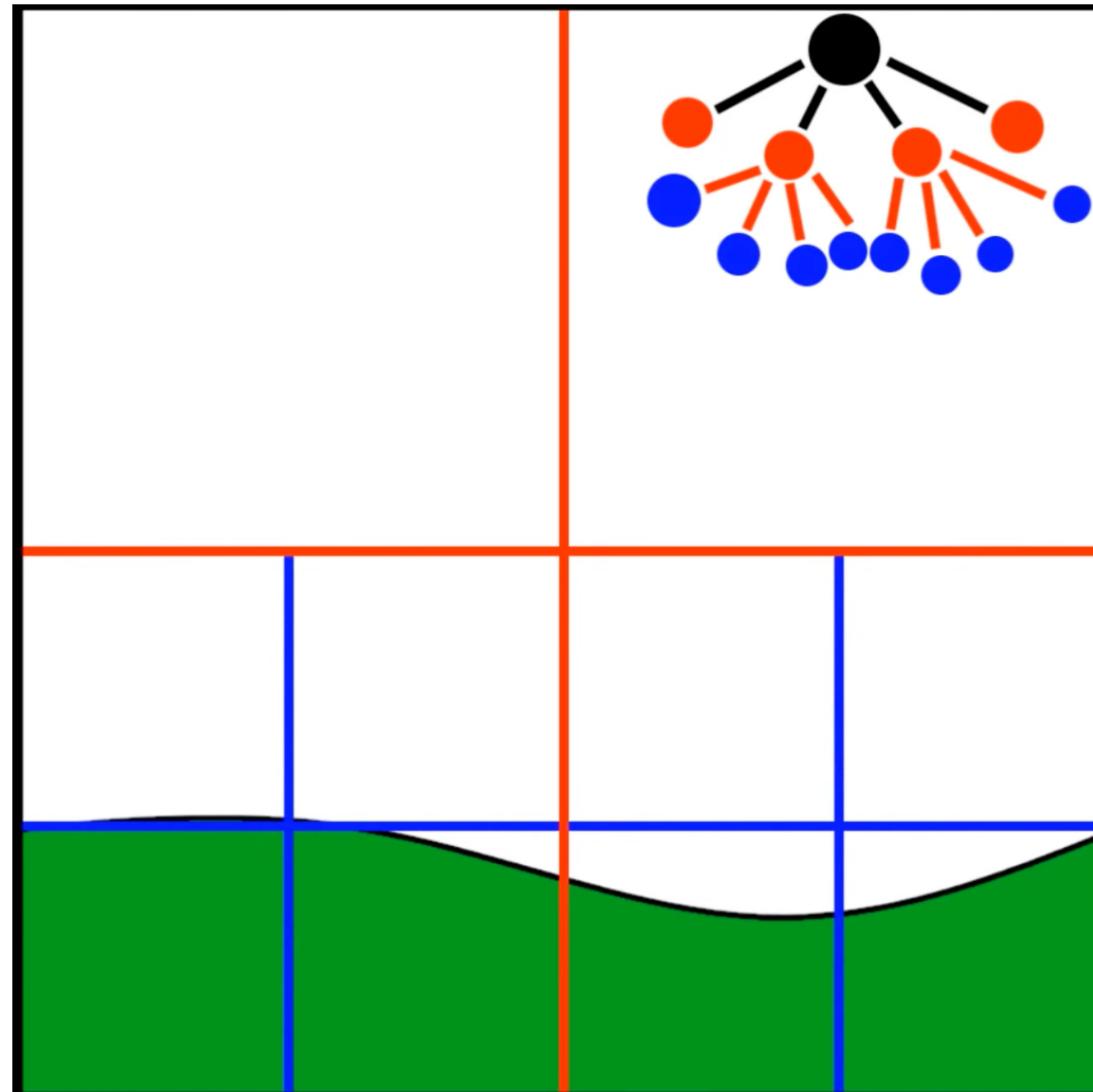


Beispiel

Das ganze Konzept lässt sich hier raus ableiten!

Der **Baum**, wie wir dies speichern könnten

Ziel: diese „landschaft“
mittels Quadtree
beschreiben



Beispiel

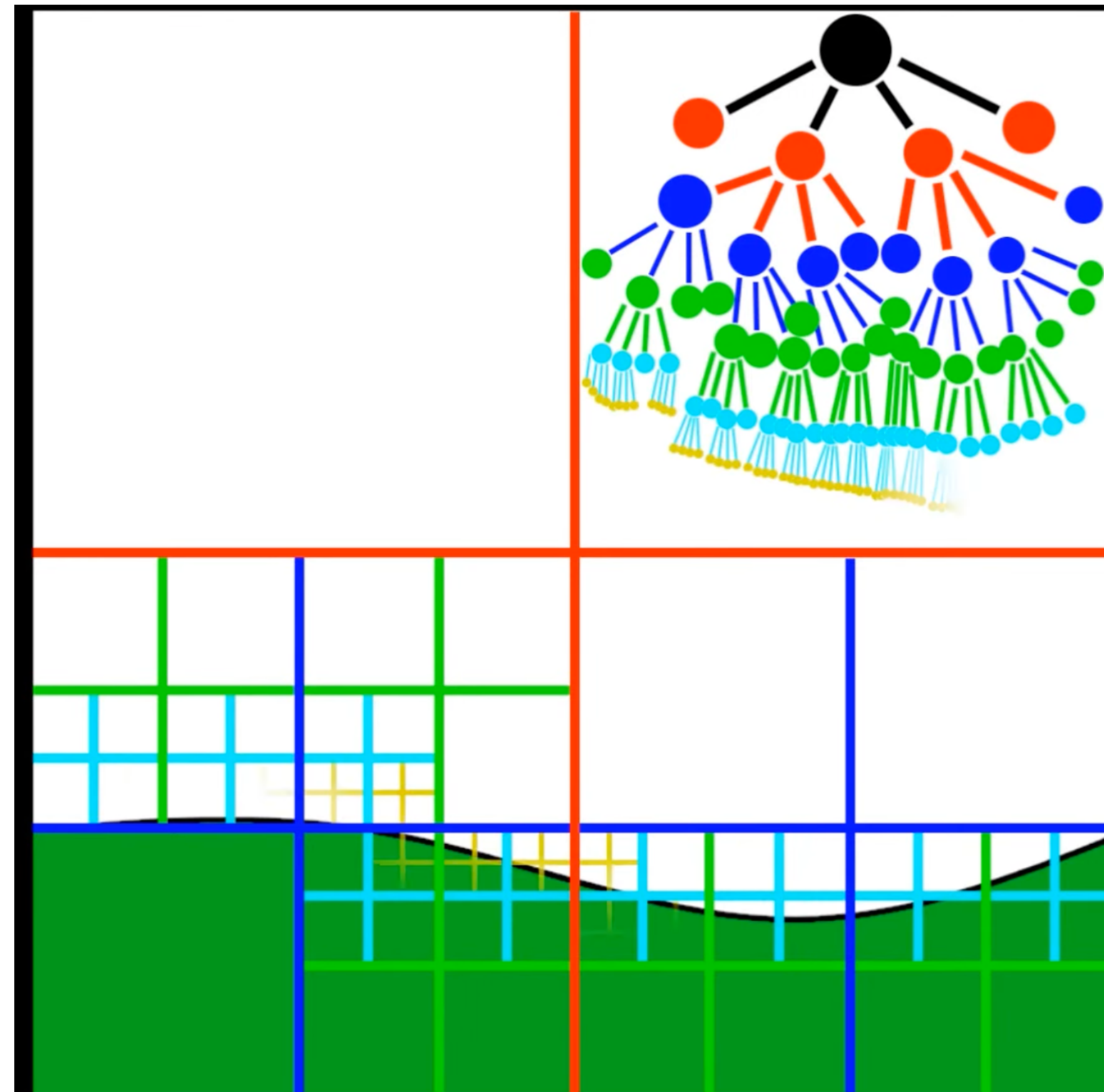
Das ganze Konzept lässt sich hier raus ableiten!

Der **Baum**, wie wir dies speichern könnten

Ziel: diese „landschaft“
mittels Quadtree
beschreiben

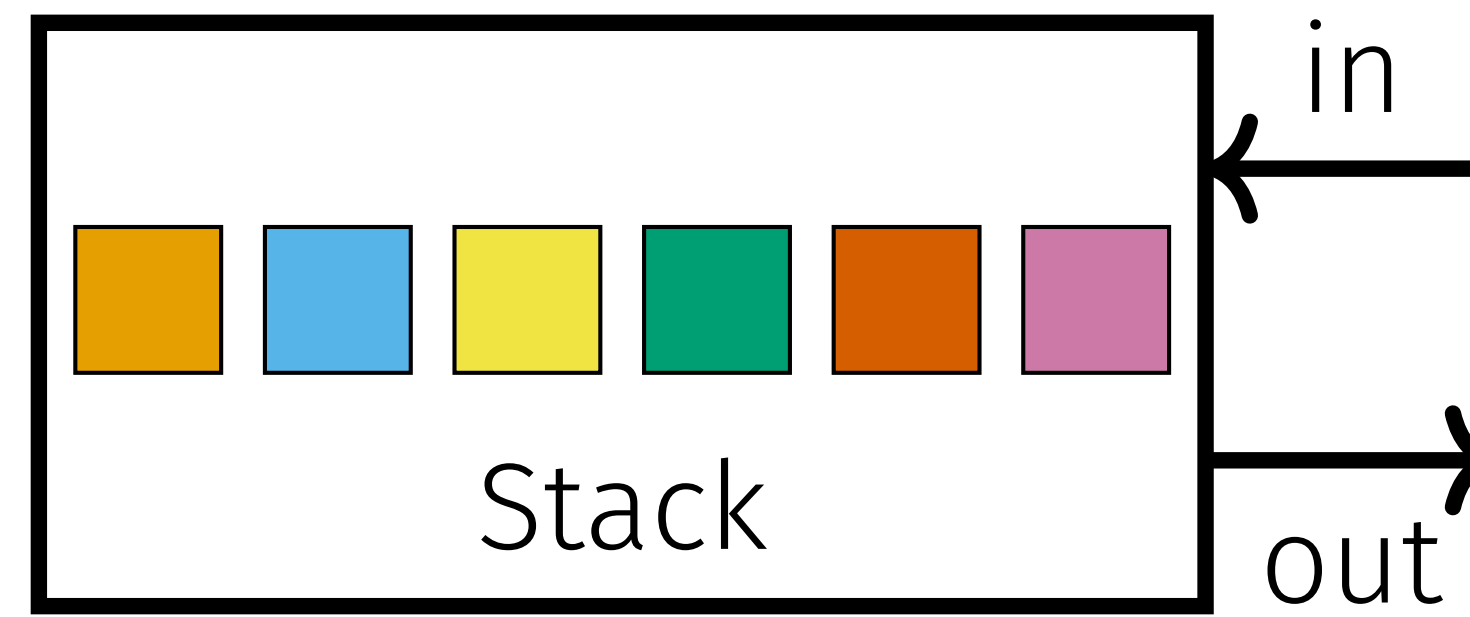
theoretische code expert
aufgabe diese woche,
genau dieses konzept

youtube.com/watch?v=jxbDYxm-pXg



Stack& Queue

Stapel (Stack): Last In First Out



Bsp. Rekursionsaufrufe!

Code Example: Einen Stack implementieren

- Welche Operationen hat ein Stack?

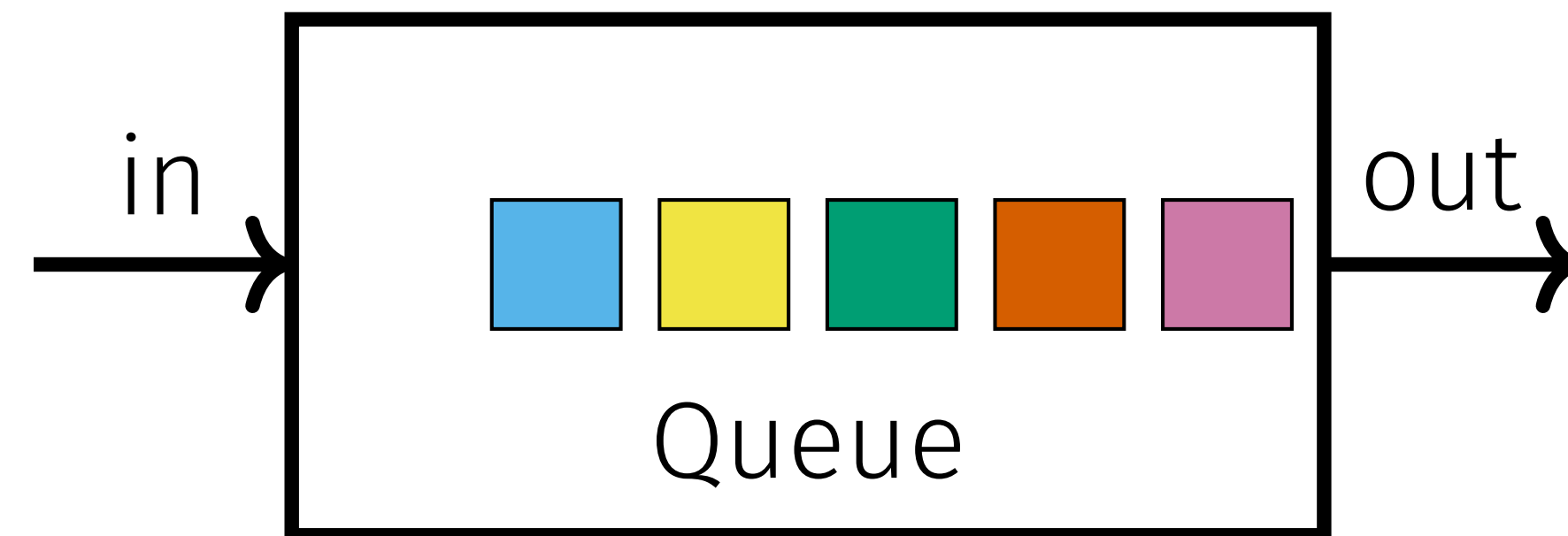
Code Example: Einen Stack implementieren

- Welche Operationen hat ein Stack?
Einfügen am Anfang, Entfernen am Anfang

Code Example: Einen Stack implementieren

- Welche Operationen hat ein Stack?
Einfügen am Anfang, Entfernen am Anfang
- Welche Datenstruktur eignet sich dafür? Welche Laufzeit?
Verkettete Liste! $\mathcal{O}(1)$

Warteschlange (Queue): First In First Out



Code Example: Queue implementieren

- Welche Operationen hat eine Queue?
Einfügen am Anfang, Entfernen am Ende
- Zu welcher Laufzeit führt die verkettete Liste?
Einfügen $O(1)$, Entfernen $O(n)$
- Welche Datenstruktur eignet sich besser dafür? Welche Laufzeit?
Doppelt verkettete Liste mit zusätzlichen Pointers in die andere Richtung!
 $O(1)$

Exercise 8: Data Structures

Bemerkungen& Tipps

- **Table Reservation System**, wichtige Aufgabe!
- **HashTable -Chainring**, gute Aufgabe
- **HashTable - Probing**
- **Quadtrees**, zwei drei Rechtecke machen, wenn man sich wohlfühlt mit dem Konzept, sollte das reichen... Etwas mühsam & trocken es für alle zu machen.

Bis nächste Woche

...nächste Woche:

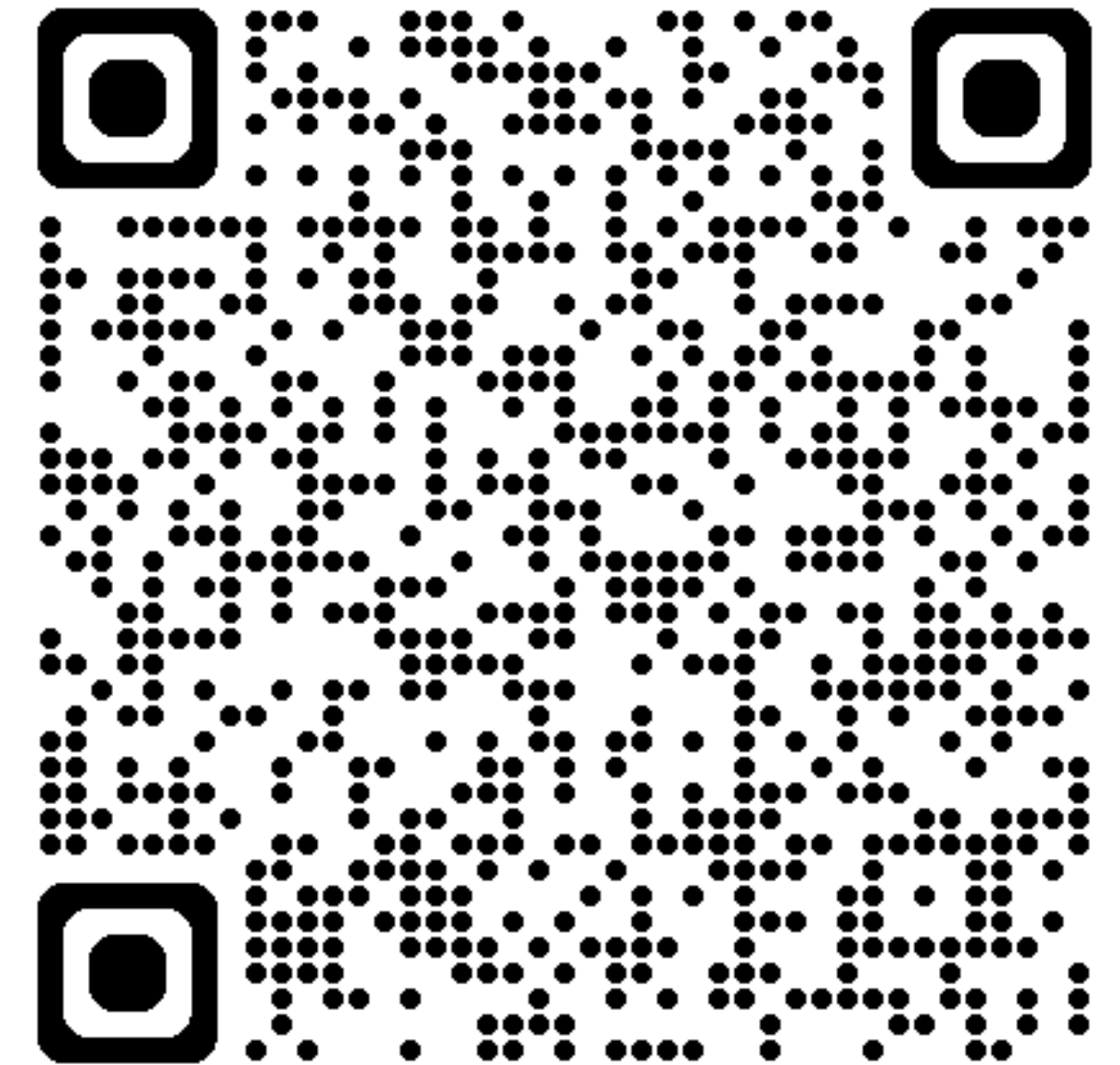
- Recap Rekursion
- **Start Dynamic Programming!**

TA evaluation, falls zufrieden, gerne voten! 🤝

(Link im Jahrgangs-Whatsapp)

nserck@ethz.ch

Anonymes Feedback Formular



Gerne Feedback: Was soll ich beibehalten/ändern

& Themenwünsche, die ich in den letzten 2 Wochen nochmals anschauen soll. (ihr könnt auch auf CX einen kommentar schreiben)